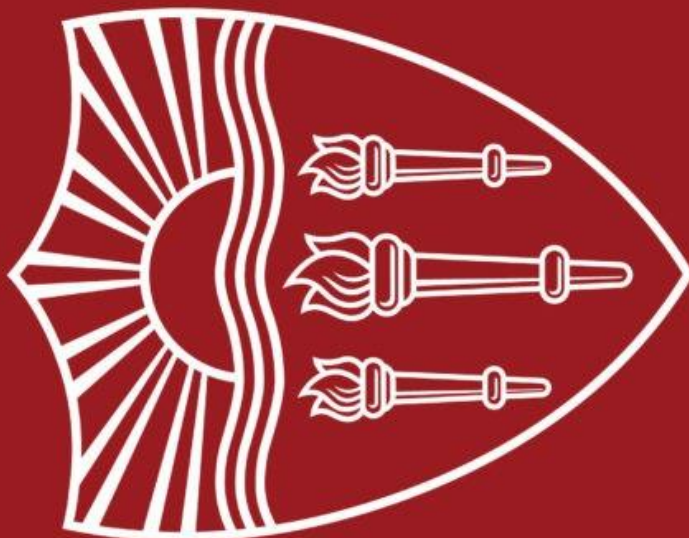




Lecture 9: Recurrent Neural Nets

Instructor: Xiang Ren
USC CSCI 544 Applied NLP
Fall 2026

Paper Review Report

- **Individual report** -- Students will write a research paper review to explain concepts underlying natural language processing in their own words.
- Two-page summary of the related work
- Options:
 1. Literature review of the course project – a few most relevant papers
 2. The course explores topics through a series of assigned readings in the form of book chapters.
- Deadline: Oct 6
- Reviews will be assessed based on answering a small set of questions, to be released at the time of the paper assignment, clearly and correctly. In most cases, each question will warrant at minimum a paragraph to answer.

Paper Review Report

1. What problem does the paper address, why does it matter, and what does it contribute? (15 points)

(5 points) **Problem setting:** clearly identify the research question or task and its relevant setting, including the inputs, outputs, or intended use where applicable.

(5 points) **Motivation:** explain why the problem matters and what limitation, uncertainty, or unmet need in existing work motivates the paper.

(5 points) **Contribution:** accurately state the main contribution and explain how it differs from prior approaches discussed in the paper or other cited sources. Distinguish the authors' novelty claims from unsupported assertions that no prior work exists.

2. How does the approach work, and why might it address the stated problem? (25 points)

(10 points) **Core approach:** provide a correct, coherent explanation of the central mechanism, procedure, or framework. For a dataset, benchmark, or analysis paper, explain its construction procedure, evaluation framework, or analytical approach rather than forcing a model-architecture description.

(10 points) **Technical details and reasoning:** explain the key components, how they interact, and why the important design choices are relevant to the problem. Describe the essential technical ideas rather than merely naming techniques; reproducing every equation or hyperparameter is not required.

(5 points) **Assumptions and trade-offs:** identify and explain at least one consequential assumption or trade-off, such as supervision requirements, computational cost, data dependence, or applicability.

Paper Review Report

3. What evidence supports the paper's claims, and what are its main findings? (25 points)

(10 points) **Evaluation setup:** identify the relevant data or setting, baselines or comparisons, and metrics or other criteria used to assess the paper's claims.

(10 points) **Main findings:** report and interpret the central findings using at least one concrete result or piece of evidence. For numerical comparisons, specify the metric, baseline, and evaluation condition. Make evidence traceable to the relevant table, figure, or section; use qualitative or theoretical evidence when appropriate.

(5 points) **Purpose of the evidence:** explain what a key comparison, ablation, analysis, or derivation is intended to establish and how it relates to a central claim. Focus on informative evidence rather than cataloging every experiment.

4. What are the paper's main strengths and limitations, and how convincing are its conclusions? (25 points)

(7 points) **Strengths:** identify at least one meaningful strength, support it with specific evidence from the paper, and explain why it matters for the paper's conclusions or usefulness.

(10 points) **Limitations:** identify at least one defensible limitation or uncertainty, support it with specific evidence or a clearly identified omission, and explain its consequences. Go beyond repeating the authors' limitations or making a generic request for more experiments.

(8 points) **Overall judgment:** provide a balanced, independently reasoned assessment of how strongly the evidence supports the central claims. Distinguish demonstrated findings, plausible interpretations, and unresolved questions; agreement or disagreement with the authors must be justified.

Paper Review Report

5. What is one worthwhile follow-up question, and how would you investigate it? (10 points)

(3 points) **Focused follow-up question:** state a specific, meaningful question grounded in the paper's findings, assumptions, or limitations.

(5 points) **Proposed investigation:** describe a plausible experiment or analysis, including the key comparison or intervention and what evidence would help answer the question. Explain why the proposed change is informative and what outcome you would examine. You do not need to run the investigation or propose a publication-level contribution.

(2 points) **Broader relevance:** explain why answering the question would improve understanding or practice in the chosen NLP topic, course concepts, or your project. A course-project connection is optional.

Lecture Outline

- Recap: Feedforward Neural Nets and Backprop
- Recurrent Neural Nets
 - Language Models
- Training RNNLMs
- The Vanishing Gradient Problem
- LSTMs
- Applications of RNNs

Recap: Feedforward Neural Nets

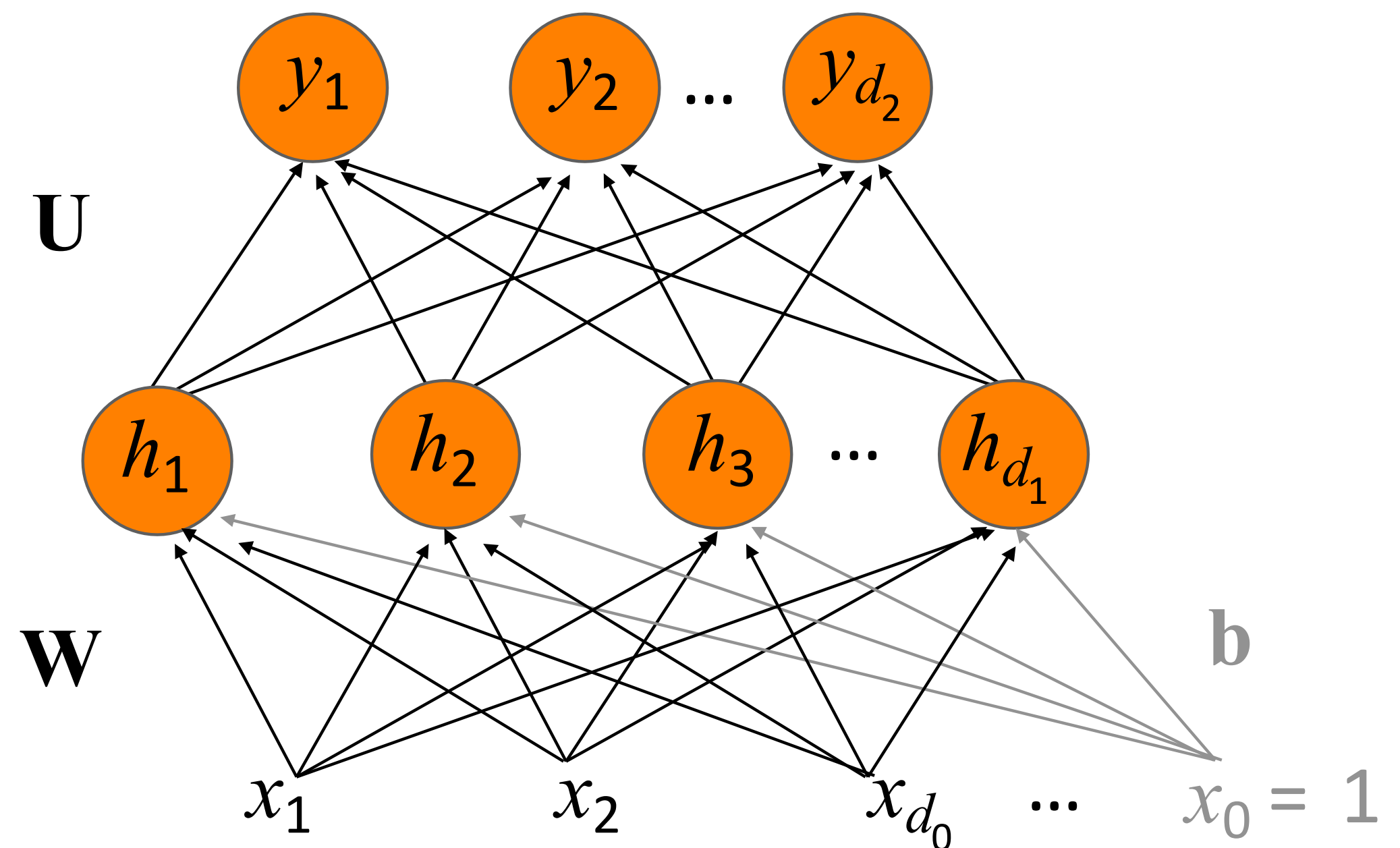
Two-layer Feedforward Network with Softmax Output

Output layer: $\mathbf{y} = \text{softmax}(\mathbf{U} \cdot \mathbf{h})$

Hidden layer: $\mathbf{h} = g(\mathbf{W}\mathbf{x} + \mathbf{b})$

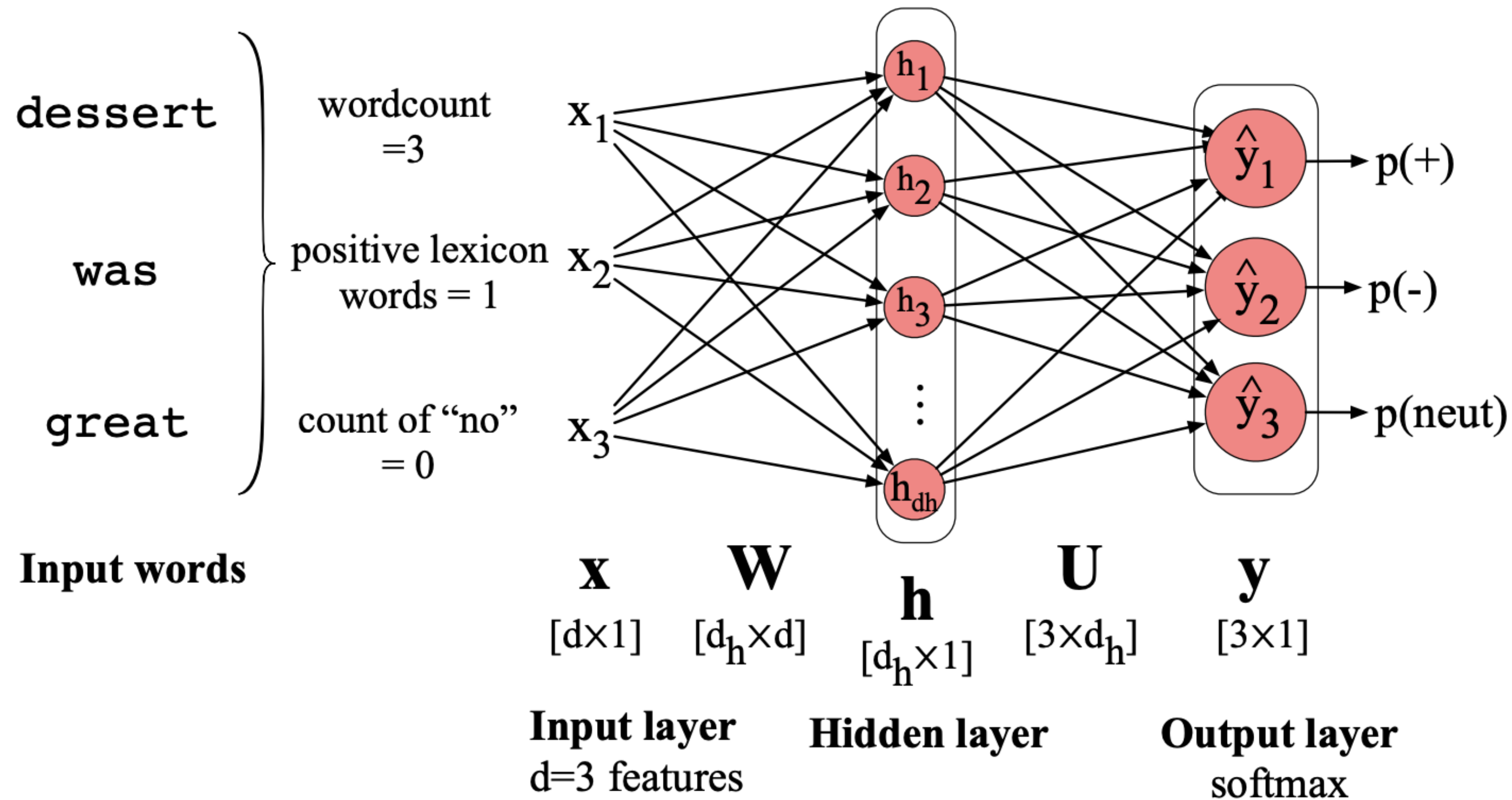
Usually ReLU or tanh

Input layer: vector



What is $\mathbf{y}$?

Sentiment Analysis



Simple Feedforward Neural LMs

Task: predict next word w_t given prior words $w_{t-1}, w_{t-2}, w_{t-3}, \dots$

Problem: Now we are dealing with sequences of arbitrary length....

Solution: Sliding windows (of fixed length)

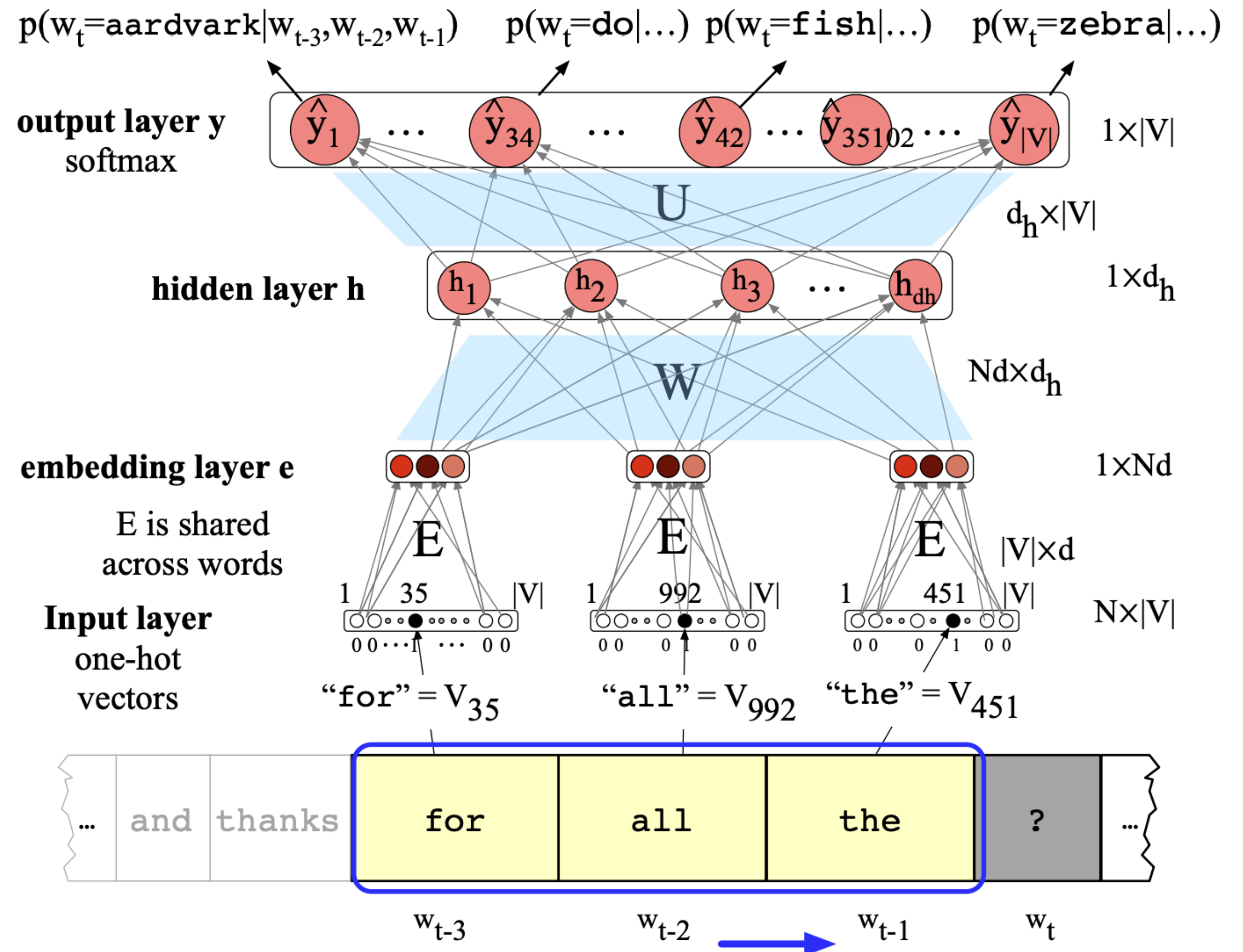
Basis of word embedding models!

$$P(w_t | w_{t-1}) \approx P(w_t | w_{t-1:t-M+1})$$

First introduced by Yoshua Bengio and colleagues in 2003

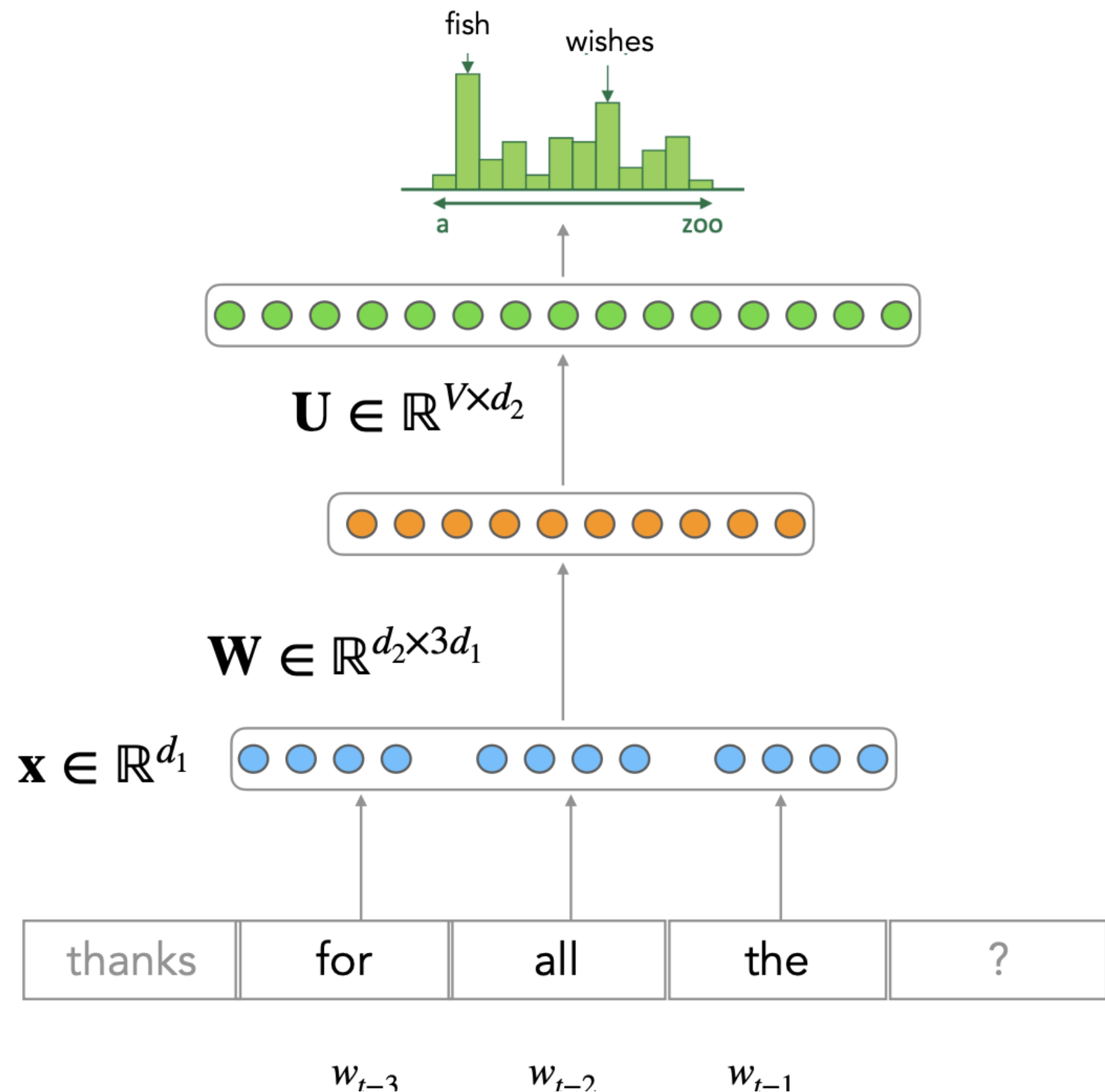
Feedforward Neural LM

- Sliding window of size 4 (including the target word)
- Every feature in the embedding vector connected to every single hidden unit
- Projection / embedding layer is a kind of input layer
 - This is where we plug in our word2vec embeddings
 - May or may not update embedding weights



Feedforward LMs: Windows

- Word embeddings $\mathbf{x}$ could either be looked up (from a dictionary of pretrained embeddings) or learned from scratch (usually with a random initialization)
- The goodness of the language model depends on the size of the sliding window
 - Fixed window can be too small
 - Enlarging window enlarges $\mathbf{W}$
 - Windows can never be large enough!
- Each word uses different rows of $\mathbf{W}$. We don't share weights *within* the window.



Reminder: Loss Function for Binary Logistic Regression

A measure for how far off the current answer is to the right answer

Cross entropy loss for logistic regression:

$$\begin{aligned} L_{\text{CE}}(\hat{y}, y) &= -\log p(y|x) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})] \\ &= -[y \log \sigma(w \cdot x + b) + (1 - y) \log(1 - \sigma(w \cdot x + b))] \end{aligned}$$

Feedforward Nets: Loss Function

- Cross Entropy Again!

$$L_{CE}(y, \hat{y}) = -\log p(y|x) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$$

From LR to FFNN: Gradients

Cross Entropy Loss for LR

$$\begin{aligned} L_{CE}(y, \hat{y}) &= -\log p(y|x) = -[y \log \hat{y} + (1-y) \log(1-\hat{y})] \\ &= -[y \log \sigma(\mathbf{w} \cdot \mathbf{x} + b) + (1-y) \log(\sigma(-\mathbf{w} \cdot \mathbf{x} + b))] \end{aligned}$$

Gradient Update for LR

$$\frac{\partial L_{CE}(\hat{y}, y)}{\partial w_j} = [\sigma(\mathbf{w} \cdot \mathbf{x} + b) - y] x_j$$

Only one parameter!

computation graphs

Typically, consider the Jacobian: $J_L = \nabla_{\mathbf{w}} L$

As (multiple) hidden layers are introduced, there will be many more parameters to consider, not to mention activation functions!

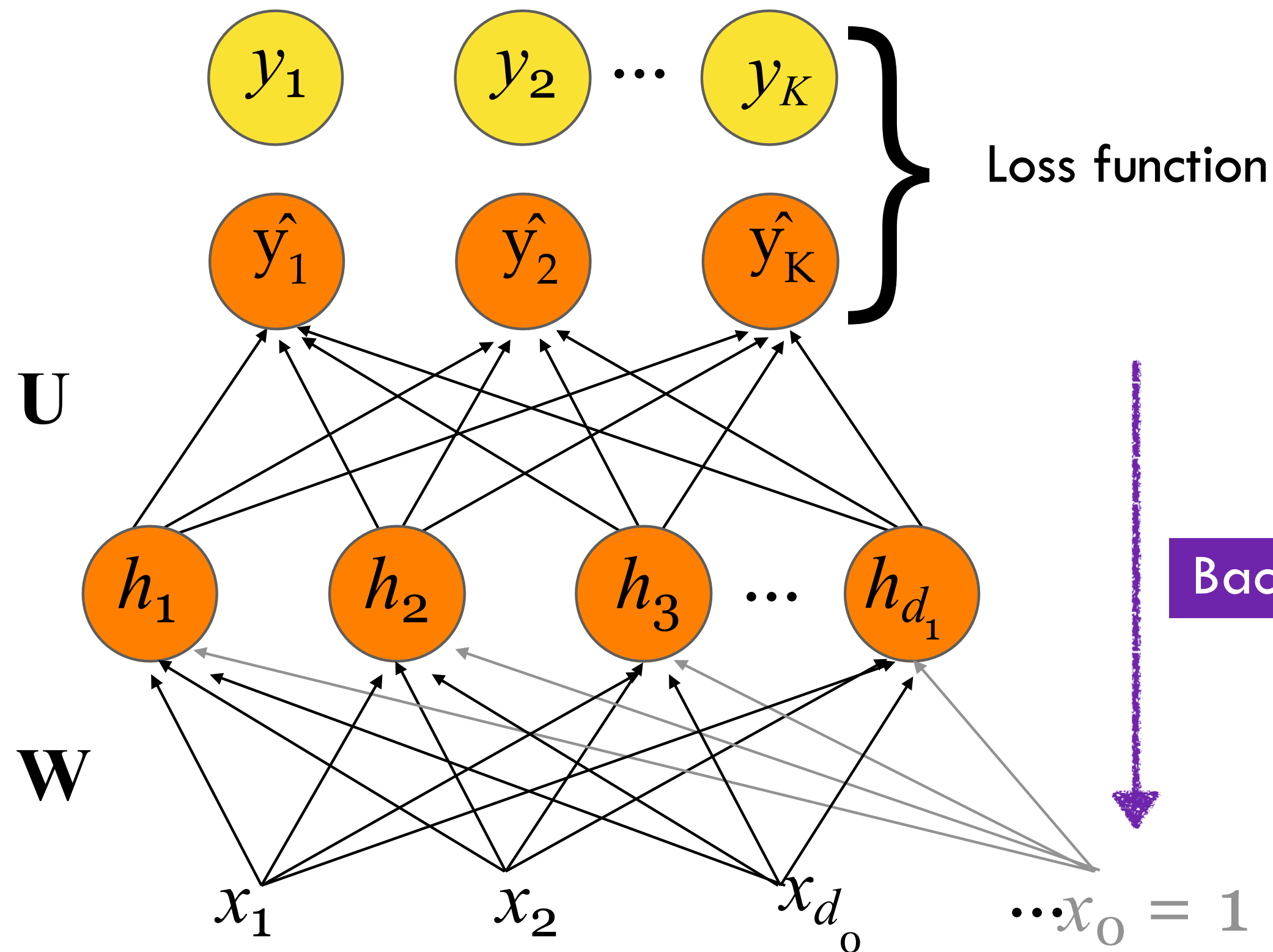
Intuition: Training a 2-layer Network

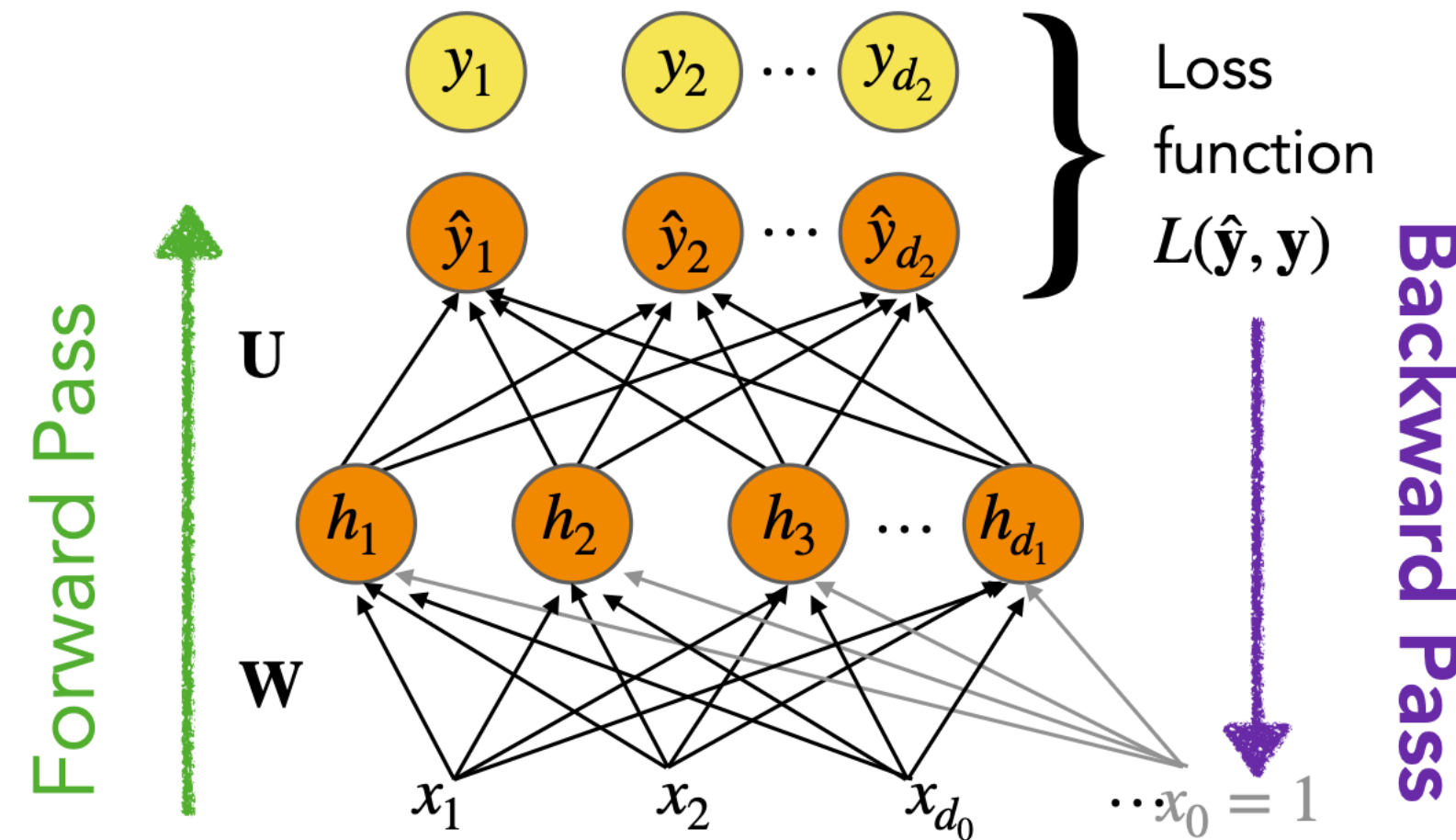
Training instance $\mathbf{y}$

Model Output $\hat{\mathbf{y}} = \text{softmax}(\mathbf{U} \cdot \mathbf{h})$

Forward Pass

Training instance





For every training tuple (x, y)

- Run **forward** computation to estimate $\hat{y}$ and compute loss L between **true y** and $\hat{y}$
- Run **backward** computation to update weights:
 - Output layer: For every weight $\mathbf{U}_{ij}$ from hidden layer to the output layer
 - Update the weight by computing gradient $\frac{\partial L}{\partial \mathbf{U}_{ij}}$
 - Hidden layer: For every weight $\mathbf{W}_{ij}$ from input layer to the hidden layer
 - Update the weight by computing gradient $\frac{\partial L}{\partial \mathbf{W}_{ij}}$

Computation Graphs

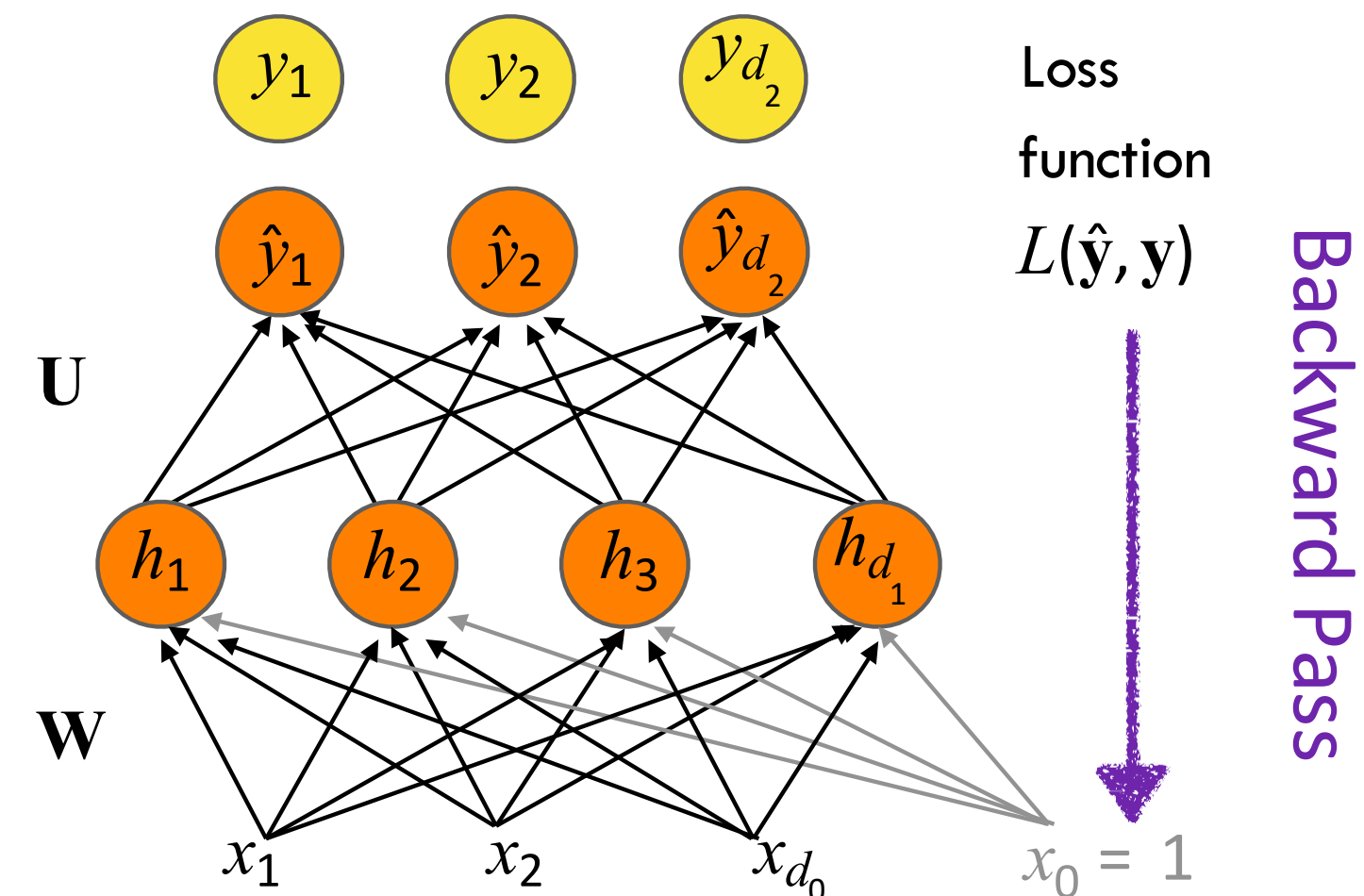
Graph representing the process of computing a mathematical expression

For training, we need the derivative of the loss with respect to each weight in every layer of the network

- But the loss is computed only at the very end of the network!

Solution: **error backpropagation** or **backward differentiation**

- Backprop is a special case of backward differentiation which relies on computation graphs



Backprop

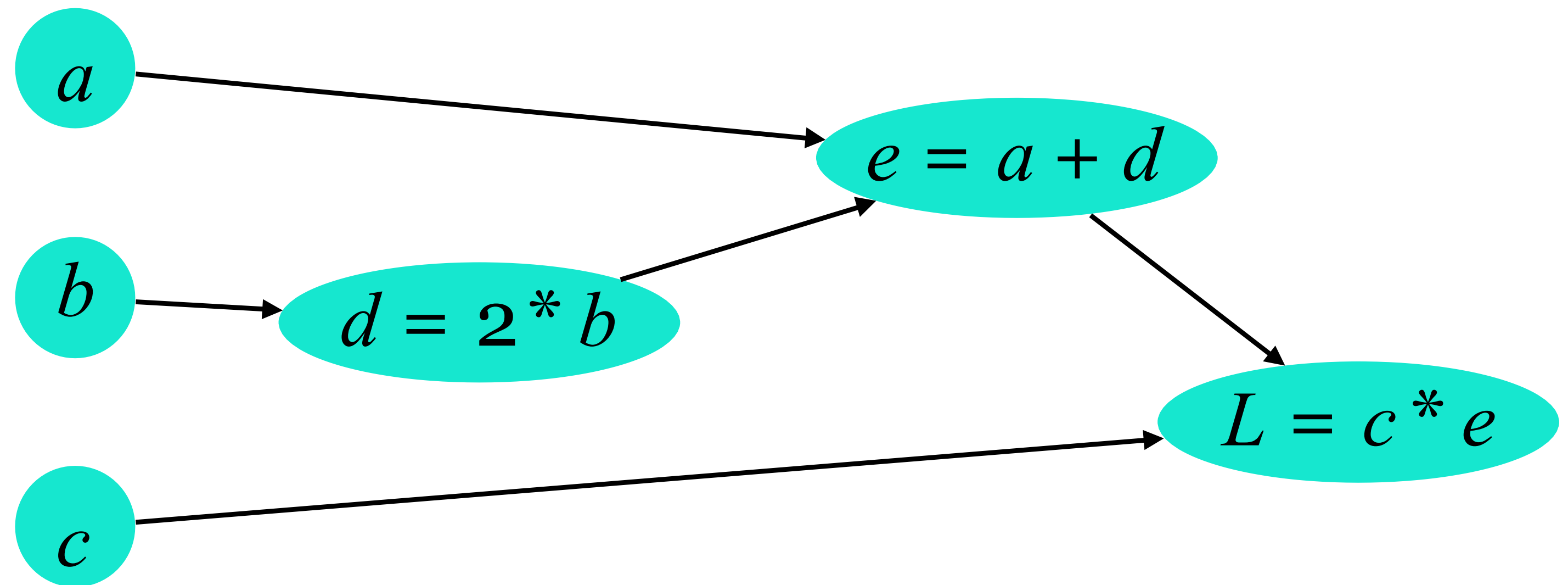
Example: Computation Graph

$$L(a, b, c) = c(a + 2b)$$

$$d = 2 * b$$

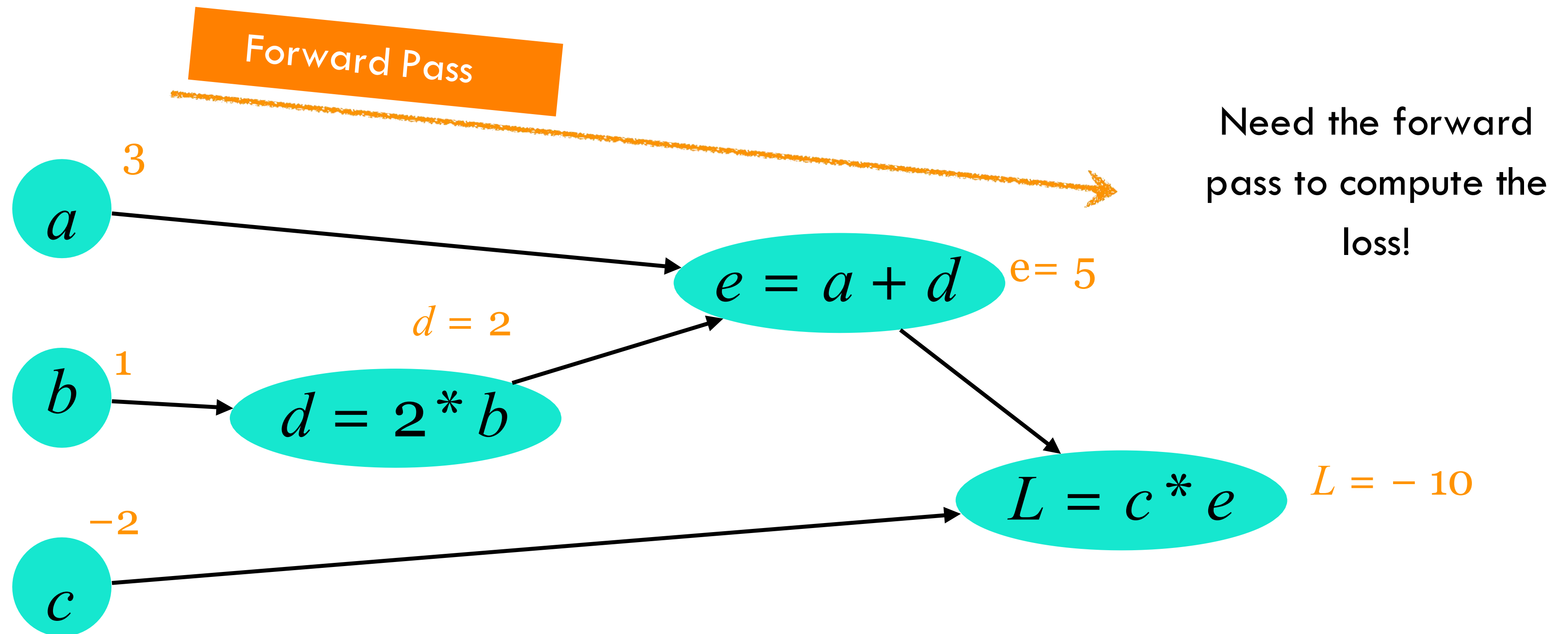
$$e = a + d$$

$$L = c * e$$



Example: Forward Pass

$$d = 2 * b$$
$$e = a + d$$
$$L = c * e$$



But how to compute parameter updates?

Example: Applying the chain rule

$$d = 2 * b$$

$$e = a + d$$

$$L = c * e$$

$$\frac{\partial L}{\partial c} = e$$

$$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial e} \frac{\partial e}{\partial a}$$

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial e} \frac{\partial e}{\partial d} \frac{\partial d}{\partial b}$$

$$\frac{\partial L}{\partial e} = c$$

$$\frac{\partial L}{\partial d} = \frac{\partial L}{\partial e} \frac{\partial e}{\partial d}$$

Cannot do all at once, need to follow an order...

Example: Backward Pass

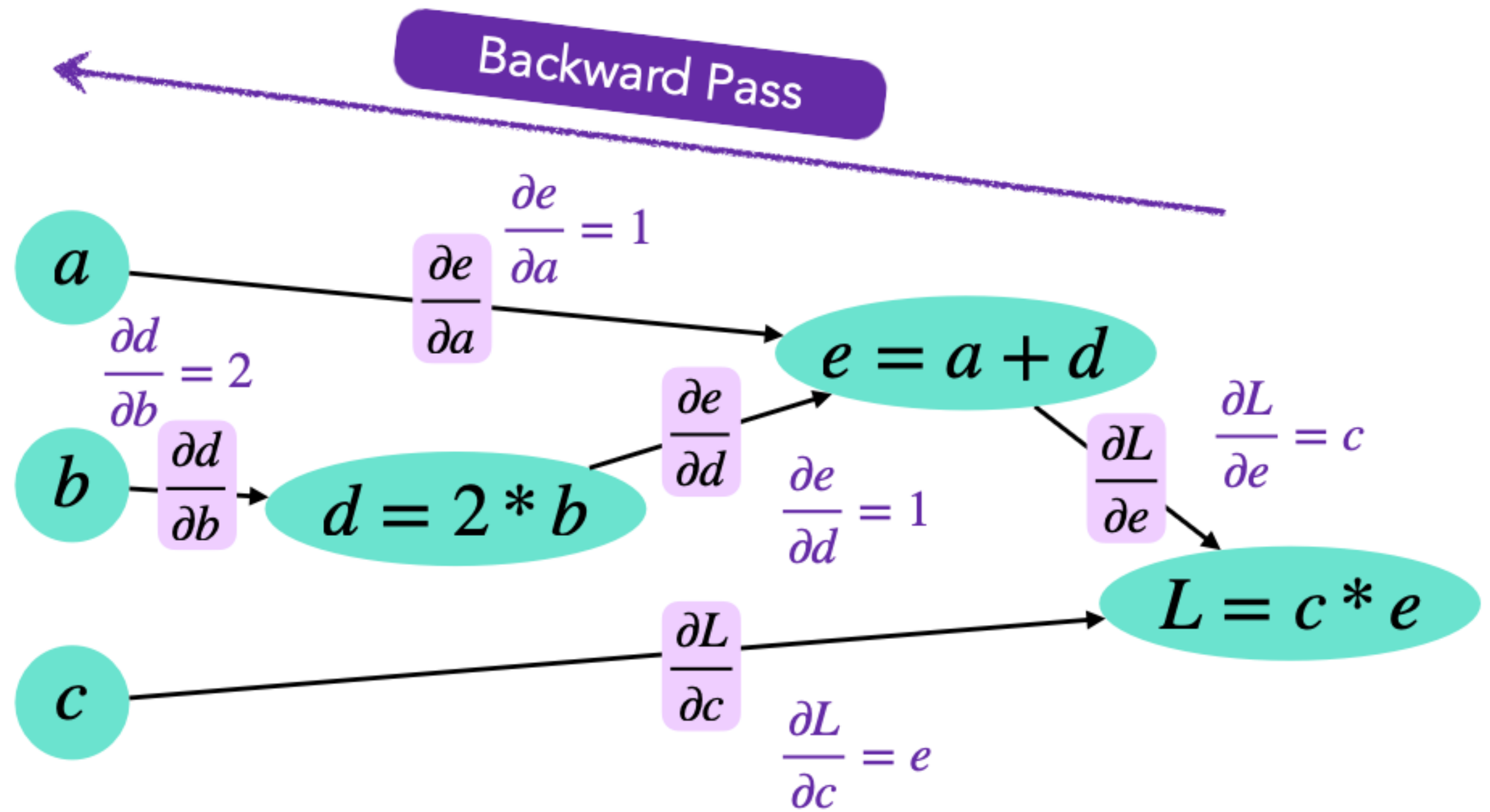
But we need the gradients of the loss with respect to parameters...

$$\frac{\partial L}{\partial c} = e \qquad \frac{\partial L}{\partial e} = c$$

$$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial e} \frac{\partial e}{\partial a}$$

$$\frac{\partial L}{\partial d} = \frac{\partial L}{\partial e} \frac{\partial e}{\partial d}$$

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial e} \frac{\partial e}{\partial d} \frac{\partial d}{\partial b}$$



Example

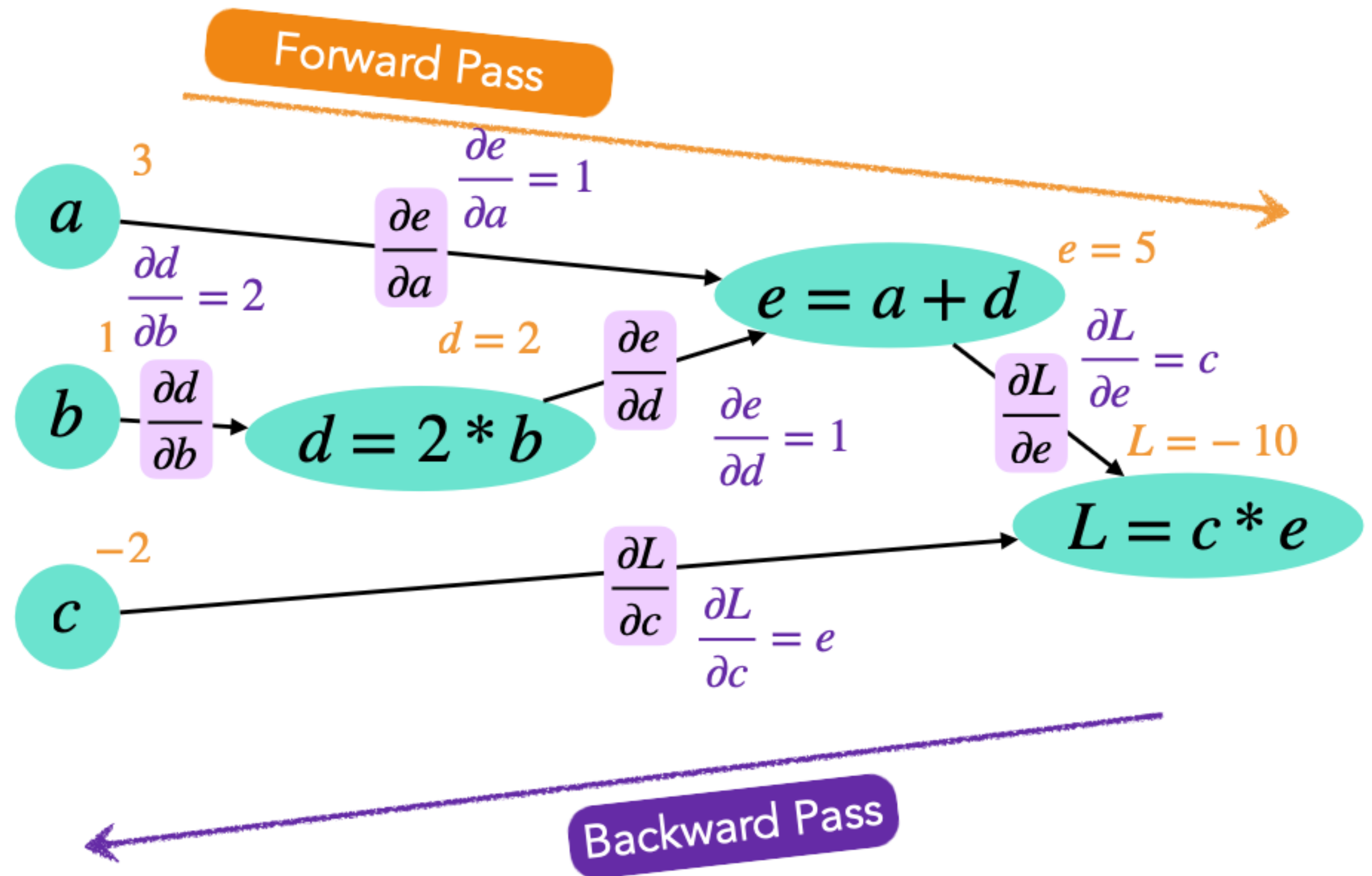
$$\frac{\partial L}{\partial e} = c = -2$$

$$\frac{\partial L}{\partial c} = e = 5$$

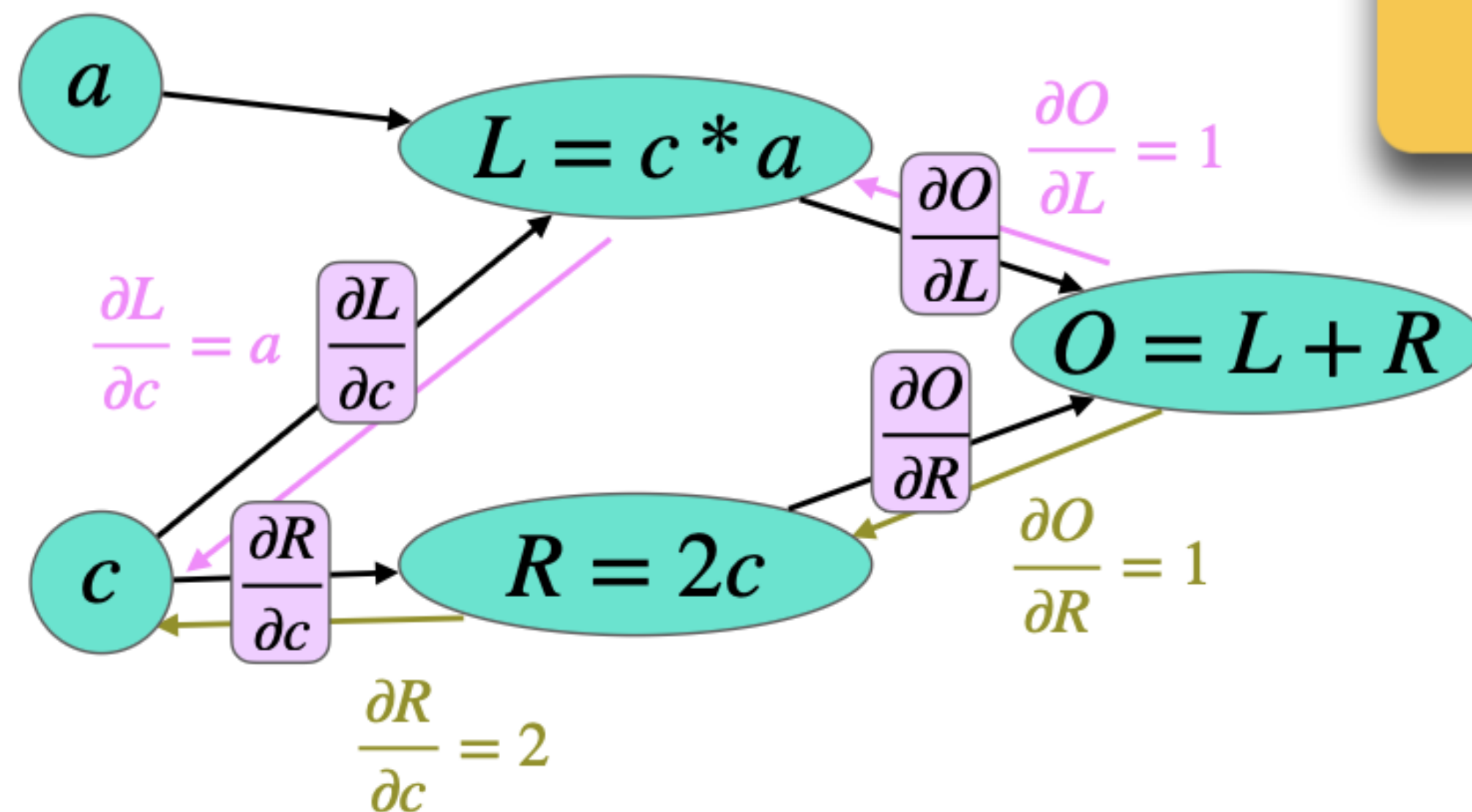
$$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial e} \frac{\partial e}{\partial a} = -2$$

$$\frac{\partial L}{\partial d} = \frac{\partial L}{\partial e} \frac{\partial e}{\partial d} = -2$$

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial e} \frac{\partial e}{\partial d} \frac{\partial d}{\partial b} = -4$$



Example: Two Paths



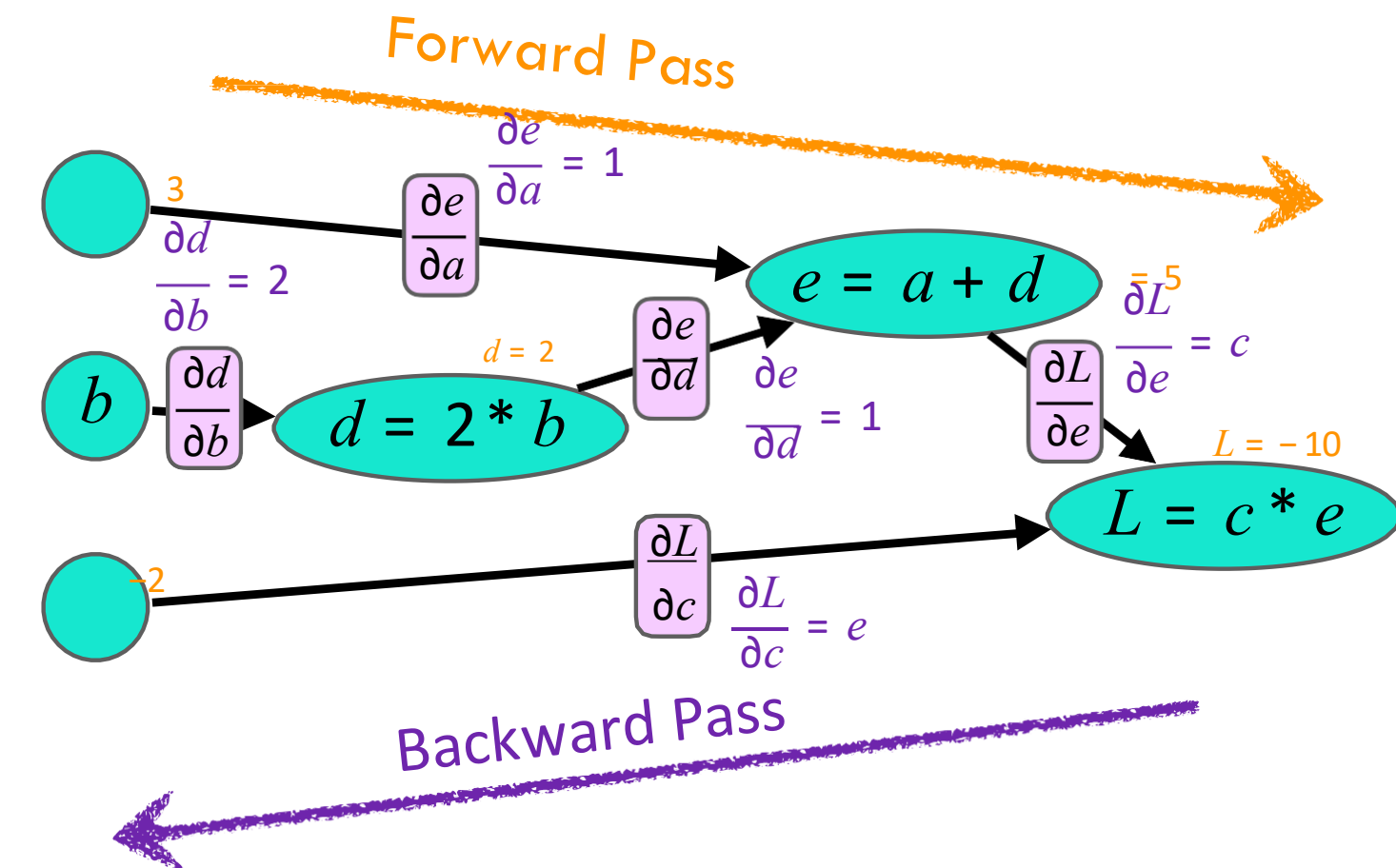
When multiple branches converge on a single node we will add these branches

$$\frac{\partial O}{\partial c} = \frac{\partial O}{\partial L} \frac{\partial L}{\partial c} + \frac{\partial O}{\partial R} \frac{\partial R}{\partial c}$$

Such cases arise when considering regularized loss functions

Backprop / Backward Differentiation

- For training, we need the derivative of the loss with respect to weights in early layers of the network
 - But loss is computed only at the very end of the network!
- Solution: backward differentiation
- Backprop is an algorithm that computes the chain rule, with a specific order of operations that is highly efficient
 - Storing repeated subexpressions, employing recursion



Given a computation graph and the derivatives of all the functions in it we can automatically compute the derivative of the loss with respect to these early weights.

Libraries such as PyTorch do this for you in a single line: `model.backward()`

Recurrent Neural Nets

Recurrent Neural Networks

- Recurrent Neural Networks processes sequences one element at a time:
 - Contains one hidden layer $\mathbf{h}_t$ per time step! Serves as a memory of the entire history...
 - Output of each neural unit at time t based both on
 - the current input at t and
 - the hidden layer from time $t - 1$
- As the name implies, RNNs have a recursive formulation
- RNNs thus don't have
 - the limited context problem that n-gram models have, or
 - the fixed context that feedforward language models have,
 - since the hidden state can in principle represent information about all of the preceding words all the way back to the beginning of the sequence

Recurrent Neural Net Language Models

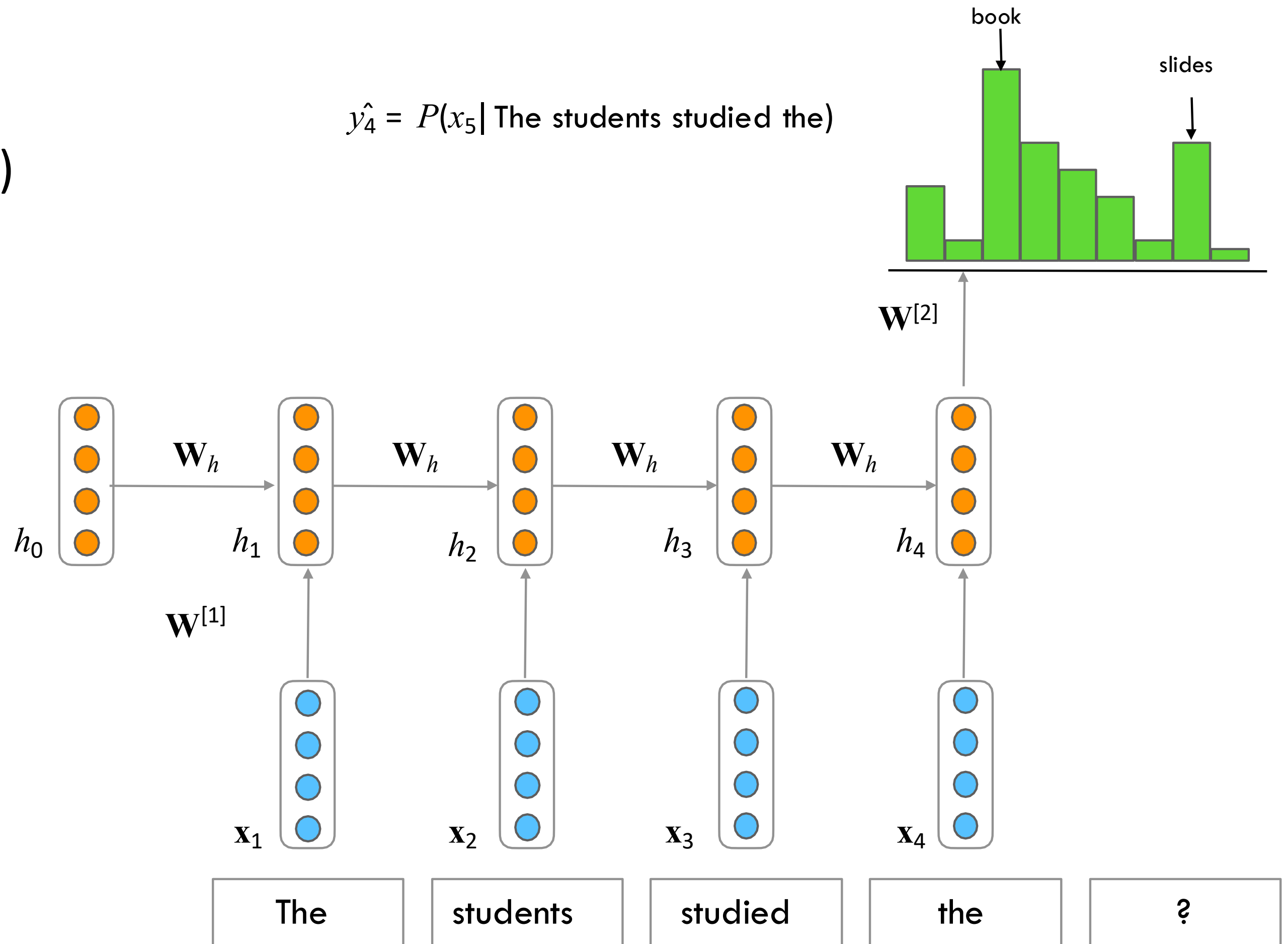
Output layer: $\hat{y}_t = \text{softmax}(\mathbf{W}^{[2]}\mathbf{h}_t)$

Hidden layer:

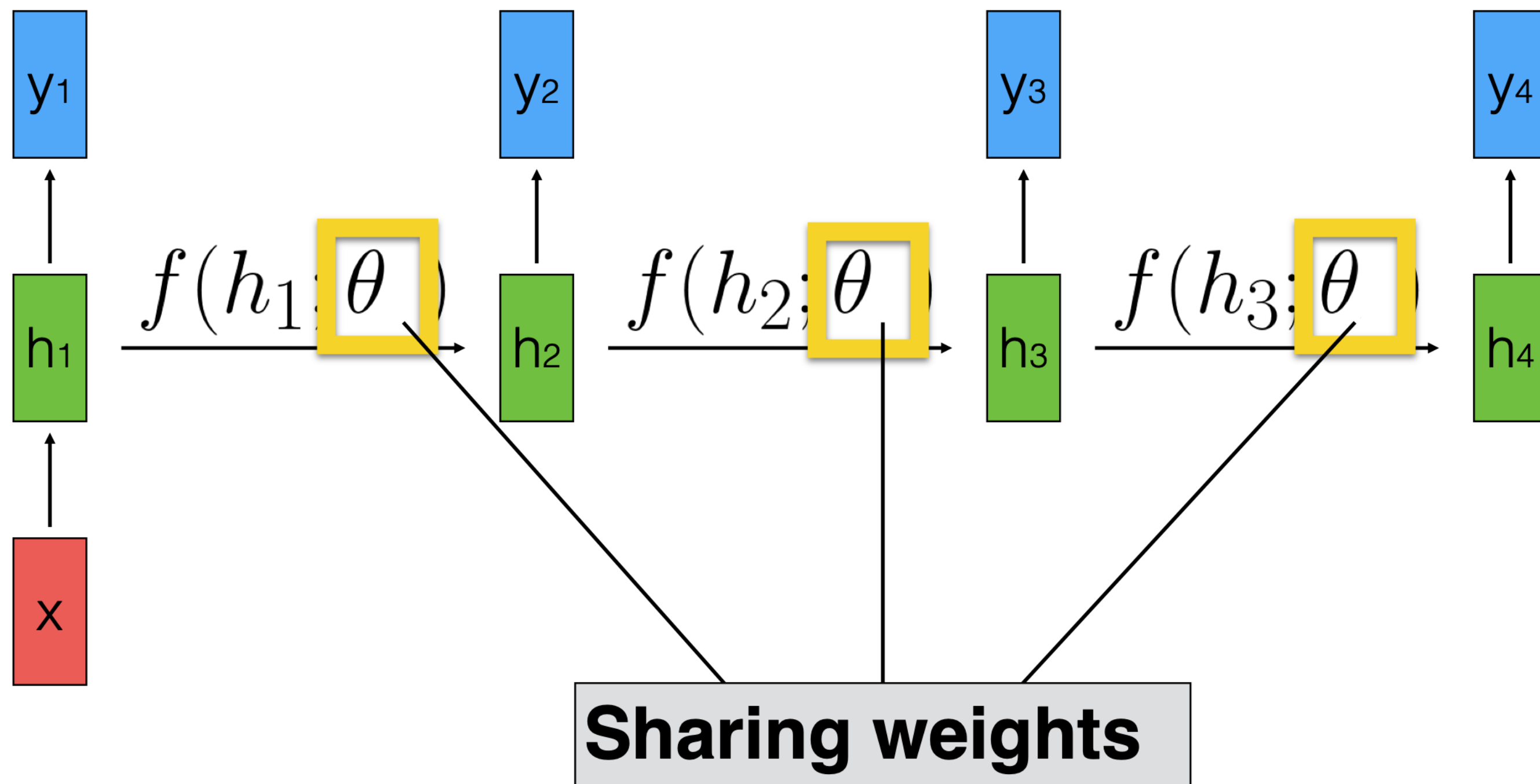
$$\mathbf{h}_t = g(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}^{[1]} \mathbf{x}_t)$$

Initial hidden state: $\mathbf{h}_0$

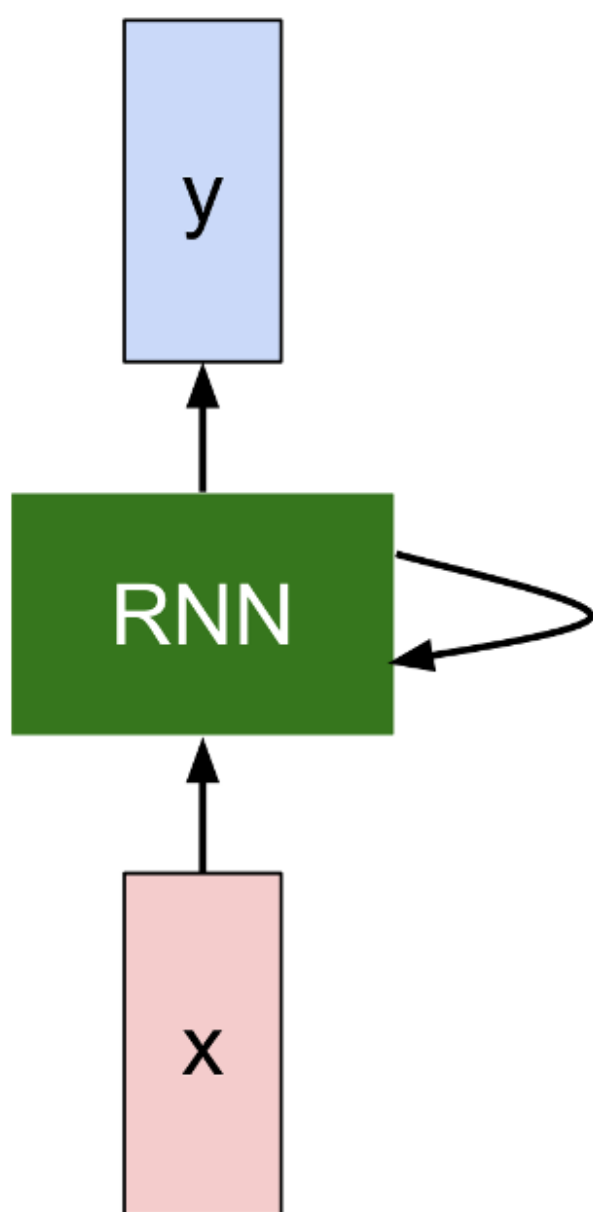
Word Embeddings, $\mathbf{x}_i$



RNN



RNN

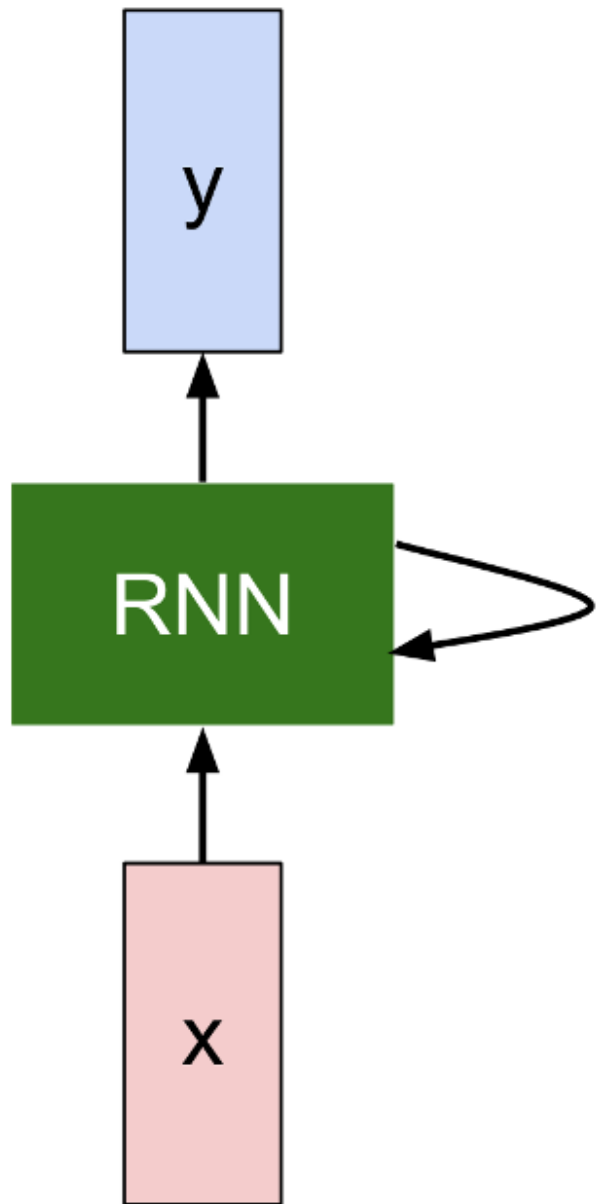


We can process a sequence of vectors $\mathbf{x}$ by applying a **recurrence formula** at every time step:

$$h_t = f_W(h_{t-1}, x_t)$$

Note: the same function with the same parameters

(Vanilla) RNN



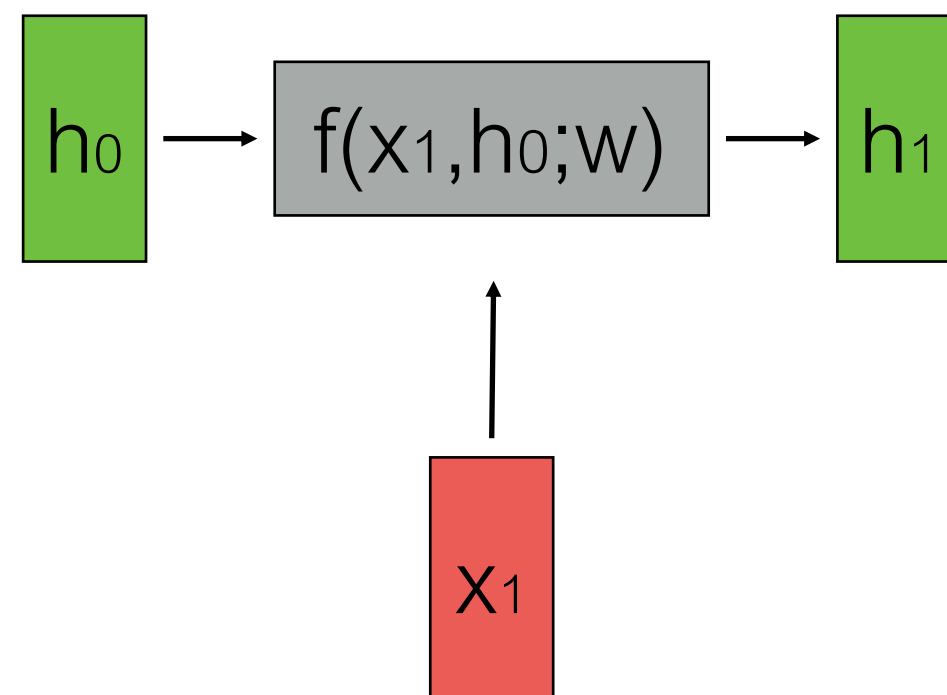
$$h_t = f_W(h_{t-1}, x_t)$$



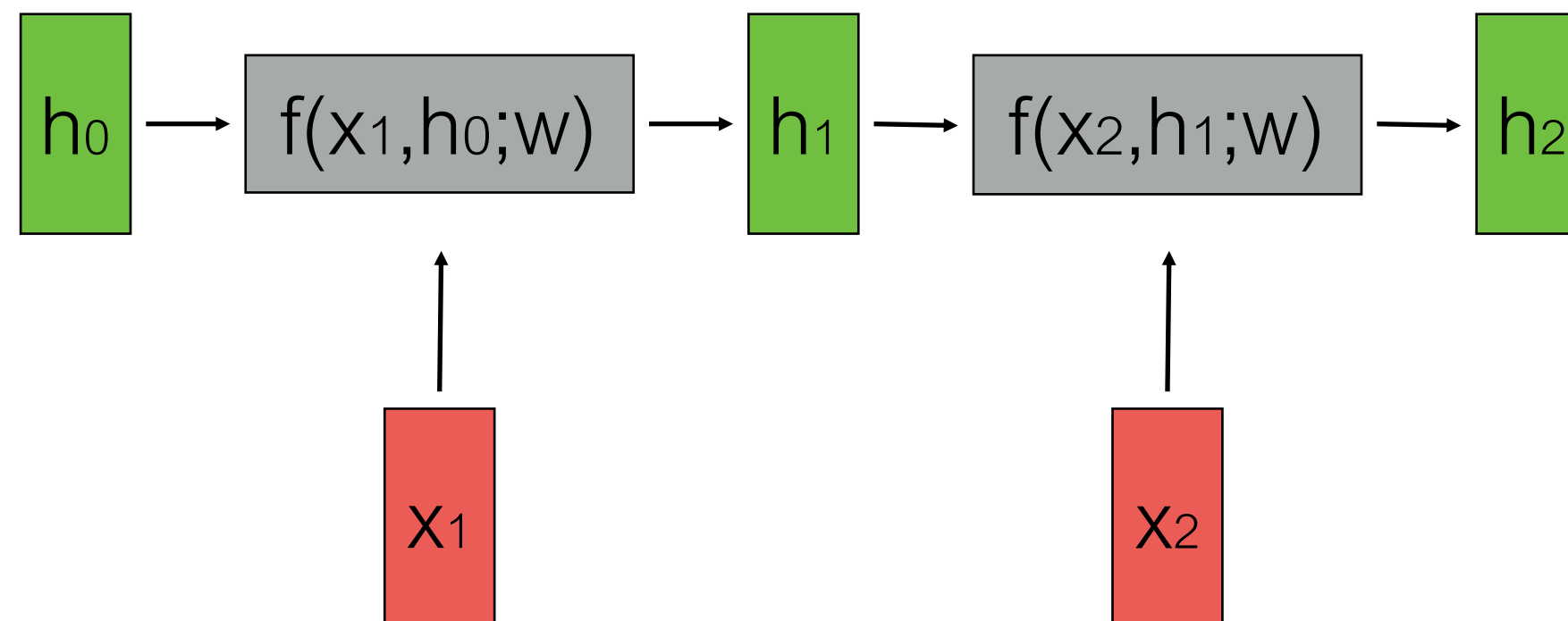
$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

$$y_t = W_{hy}h_t$$

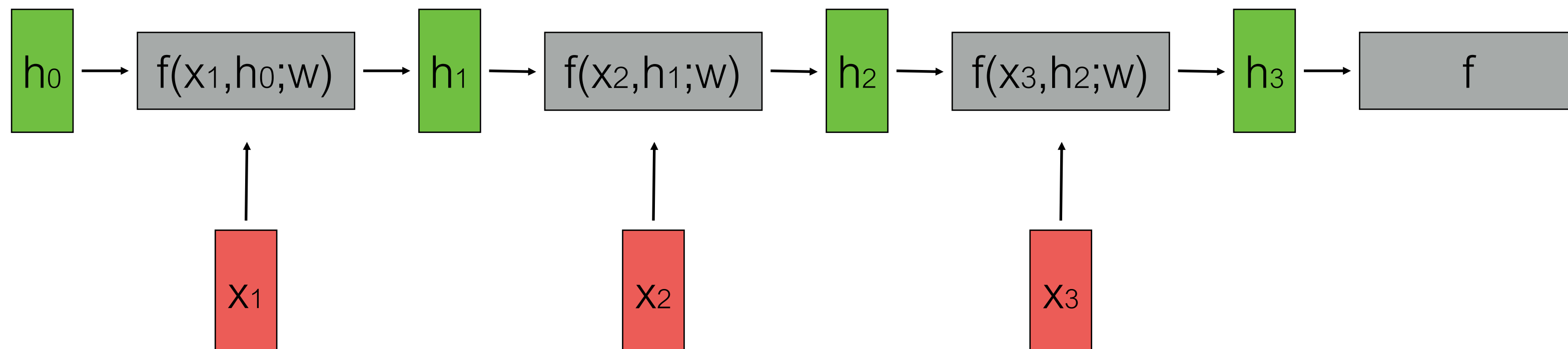
Computational Graph for RNN



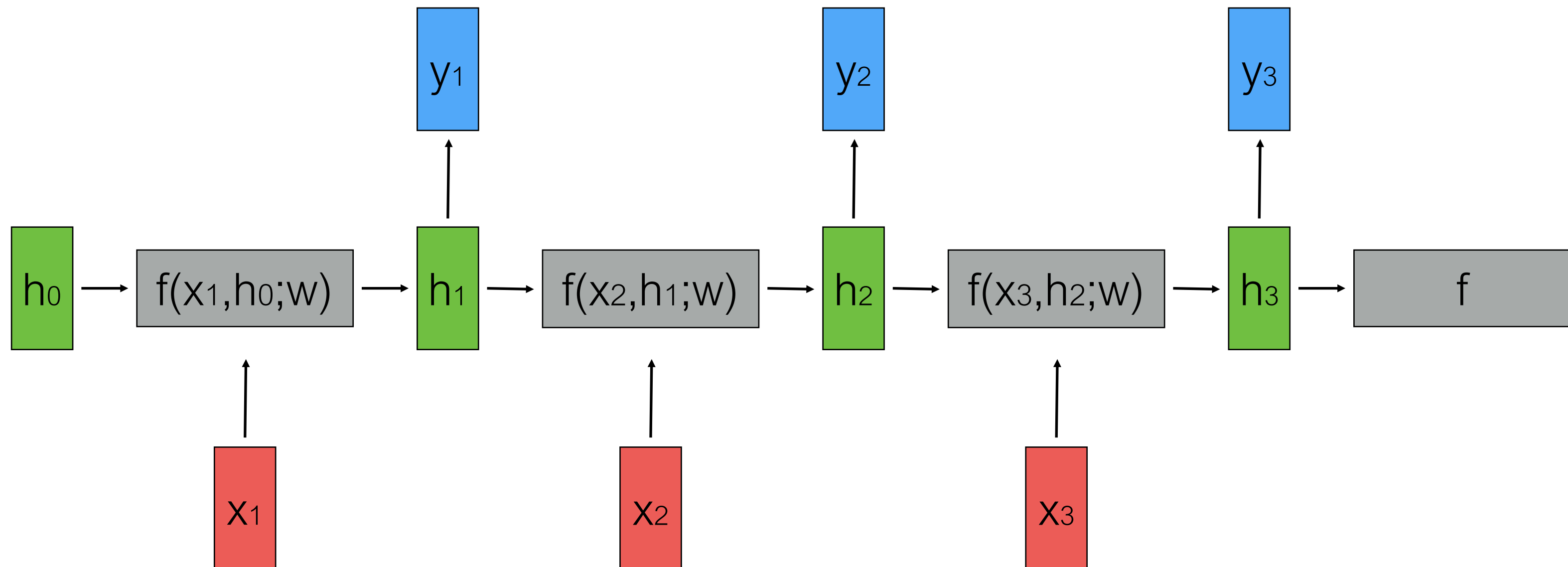
Computational Graph for RNN



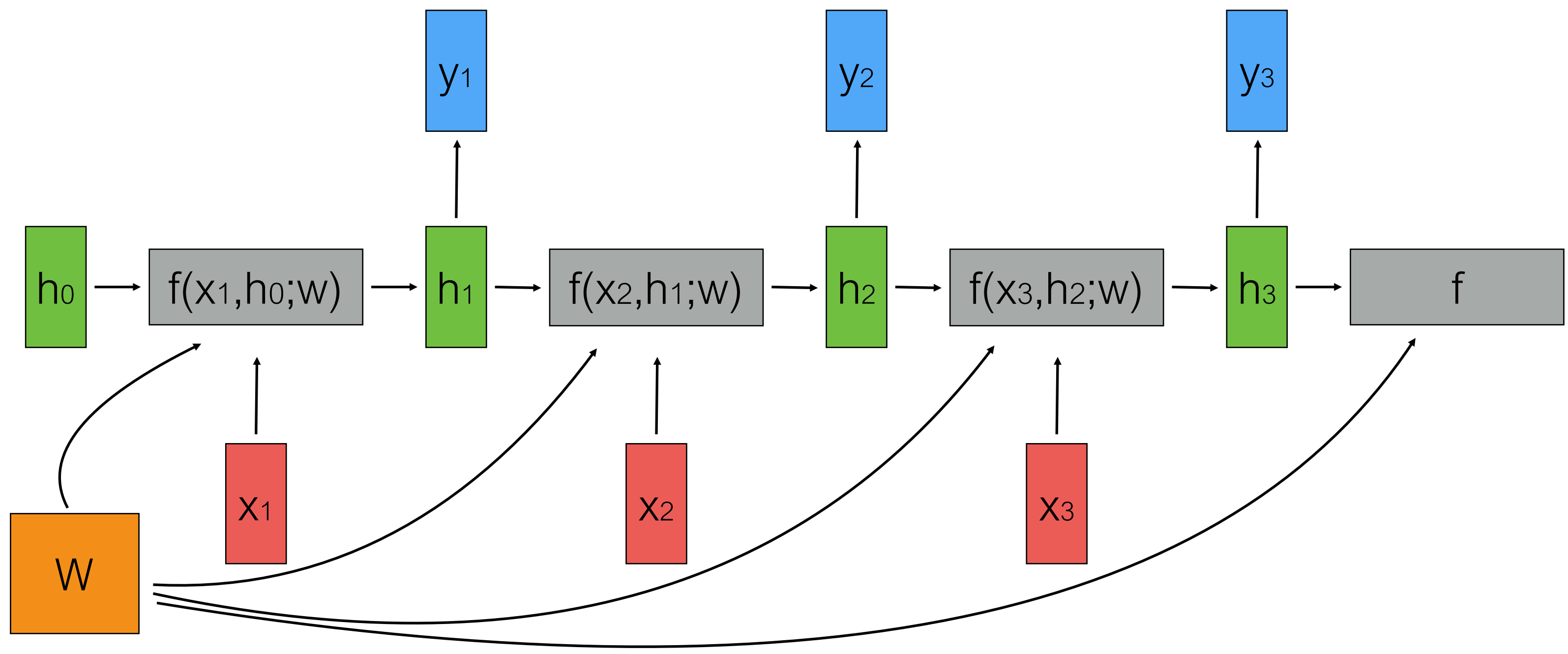
Computational Graph for RNN



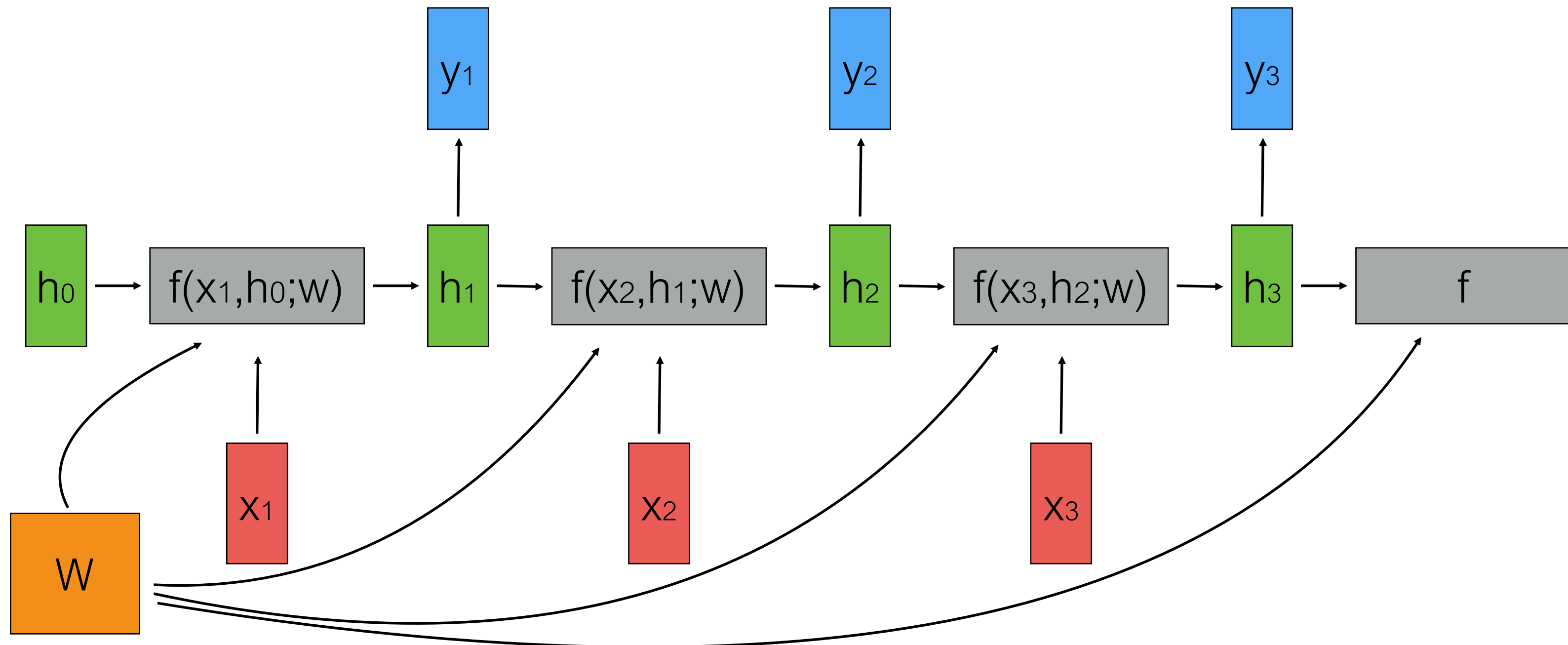
Computational Graph for RNN



Computational Graph for RNN

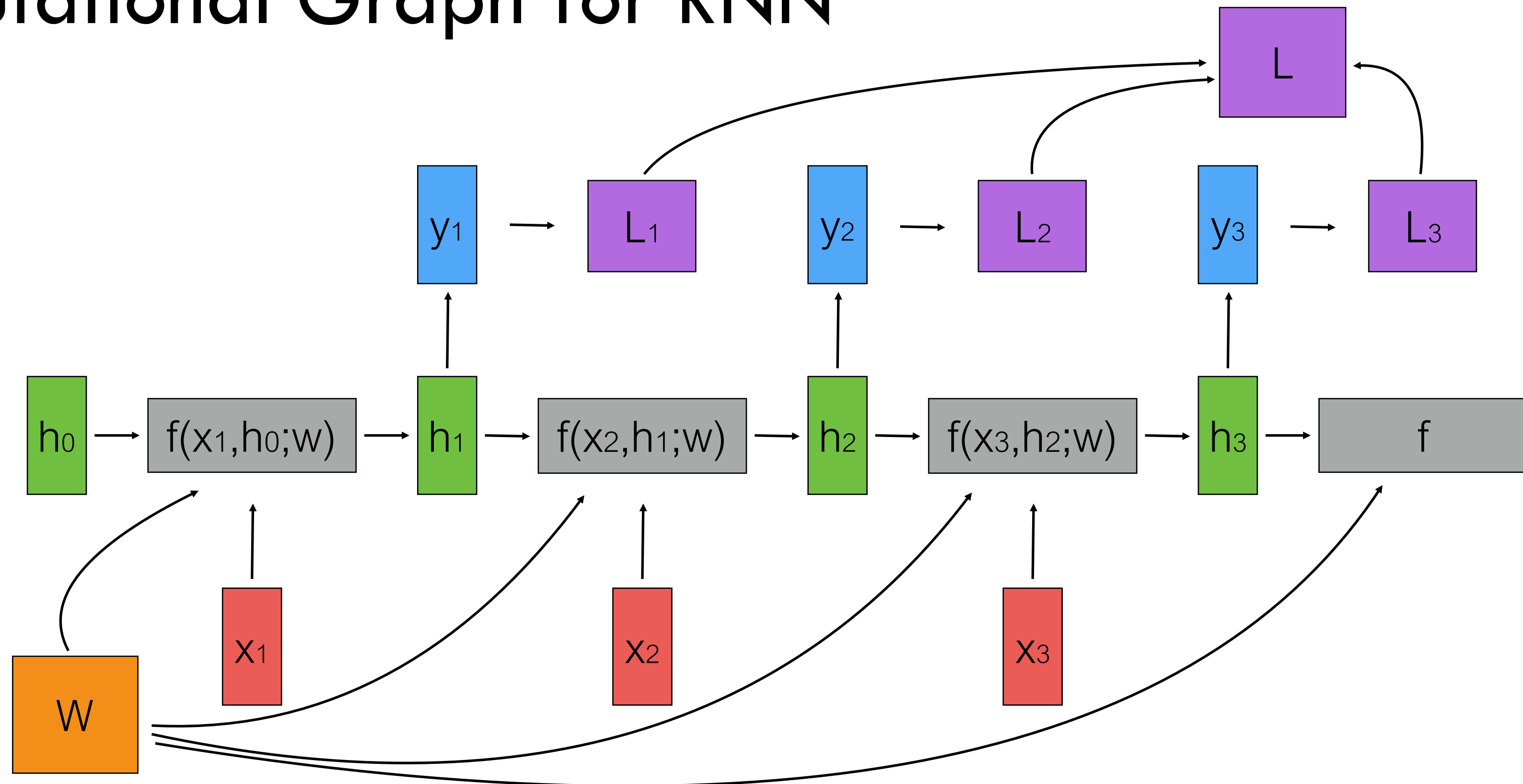


Computational Graph for RNN



The same weight (w) is shared across all f

Computational Graph for RNN

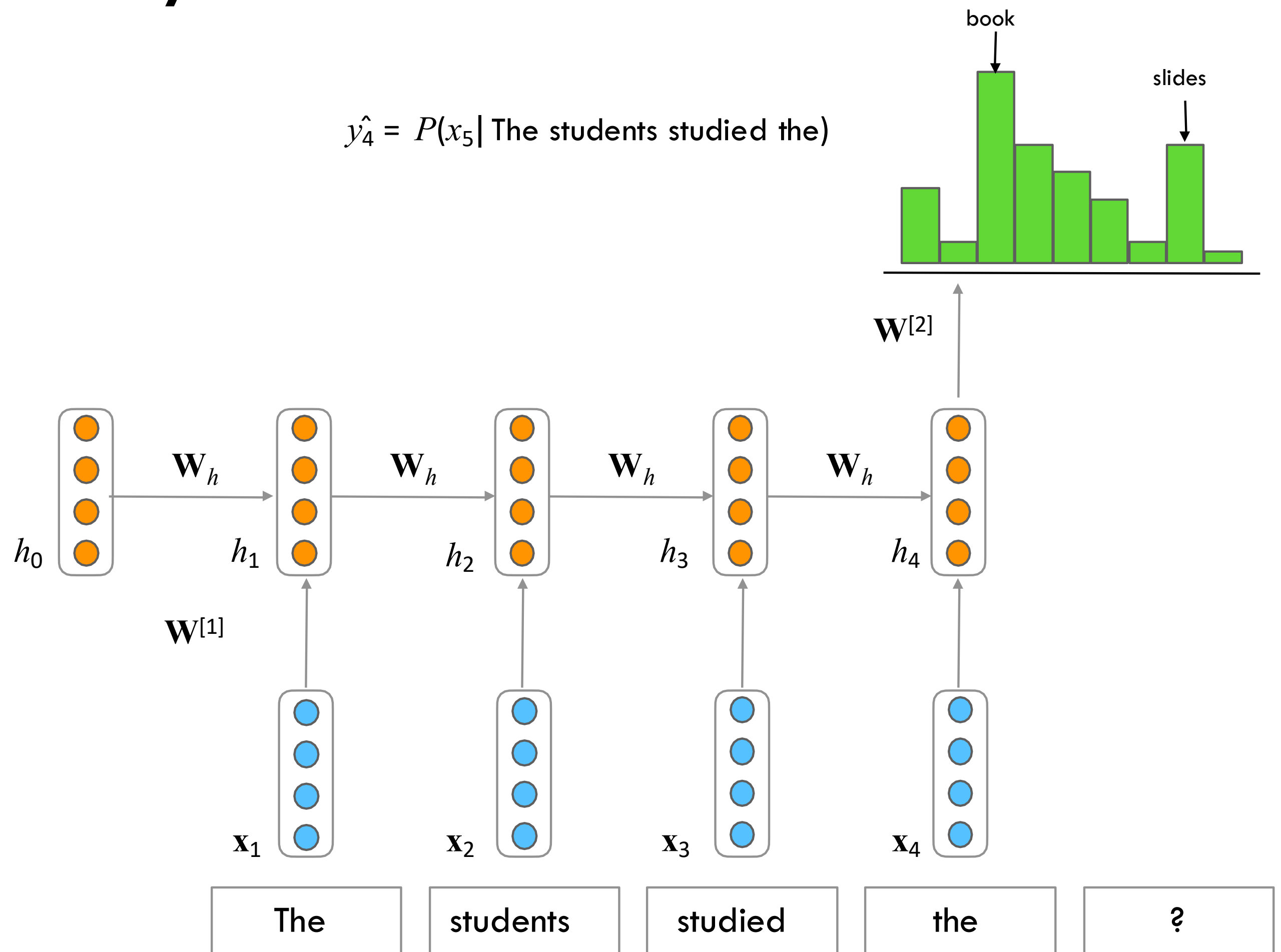


The same weight (w) is shared across all f

Why RNNs?

RNN Advantages:

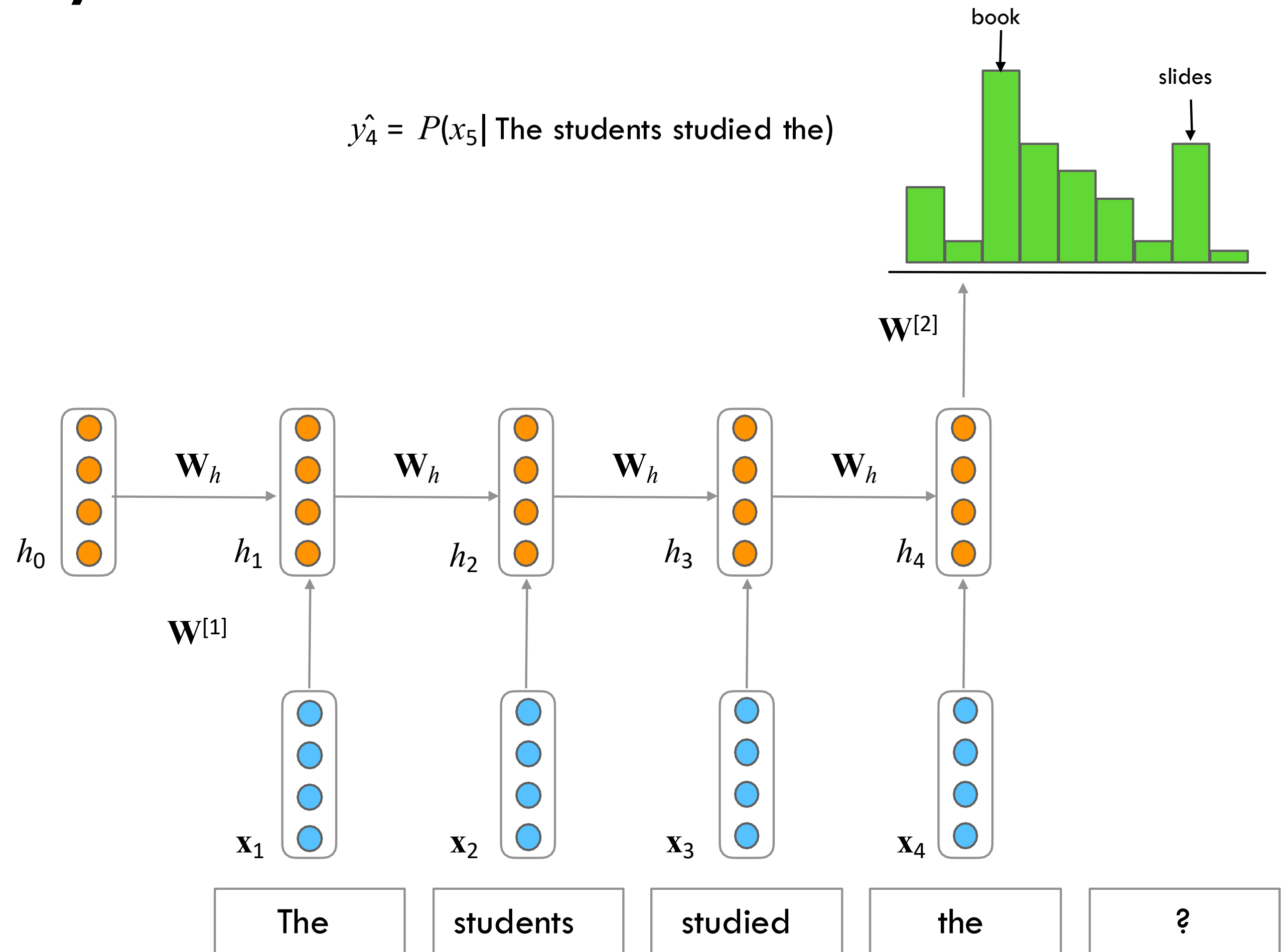
- Can process any length input
- Model size doesn't increase for longer input
- Computation for step t can (in theory) use information from many steps back
- Weights $\mathbf{W}^{[1]}$ are shared (tied) across timesteps $\rightarrow$ Condition the neural network on all previous words



Why not RNNs?

RNN Disadvantages:

- Recurrent computation is slow
- In practice, difficult to access information from many steps back



Training RNNLMs

Training Outline

- Get a big corpus of text which is a sequence of words $x_1, x_2, \dots, x_T$
- Feed into RNN-LM; compute output distribution $\hat{y}_t$ for every step
 - i.e. predict probability distribution of every word, given words so far
- Loss function on step t is usual cross-entropy between our predicted probability distribution $\hat{y}_t$ and the true next word $y_t = x_{t+1}$:

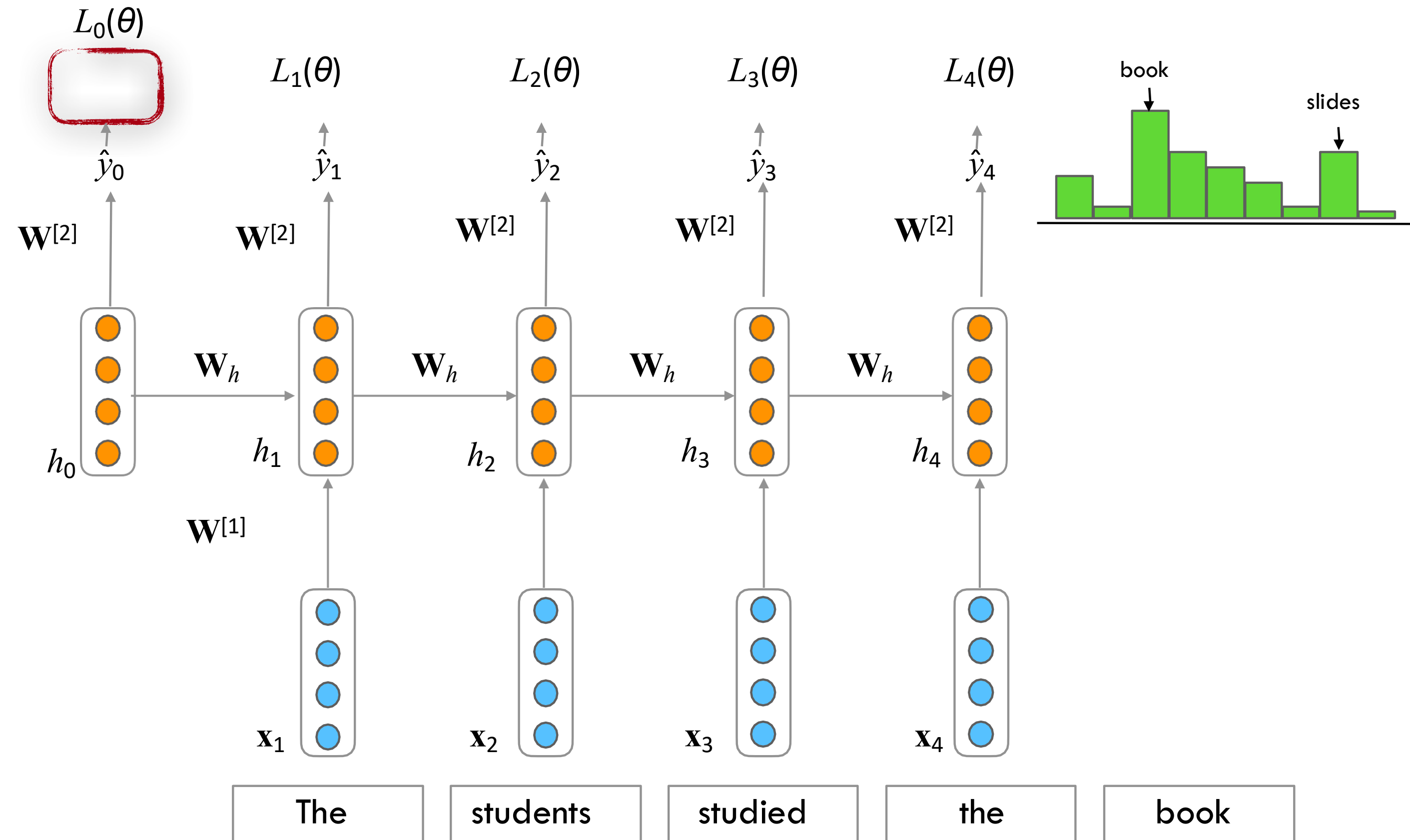
$$L_{CE}(\hat{y}_t, y_t; \theta) = - \sum_{v \in V} \mathbb{I}[y_t = v] \log \hat{y}_t[v] = - \log p_\theta(x_{t+1} | x_{\leq t})$$

- Average this to get overall loss for entire training set:

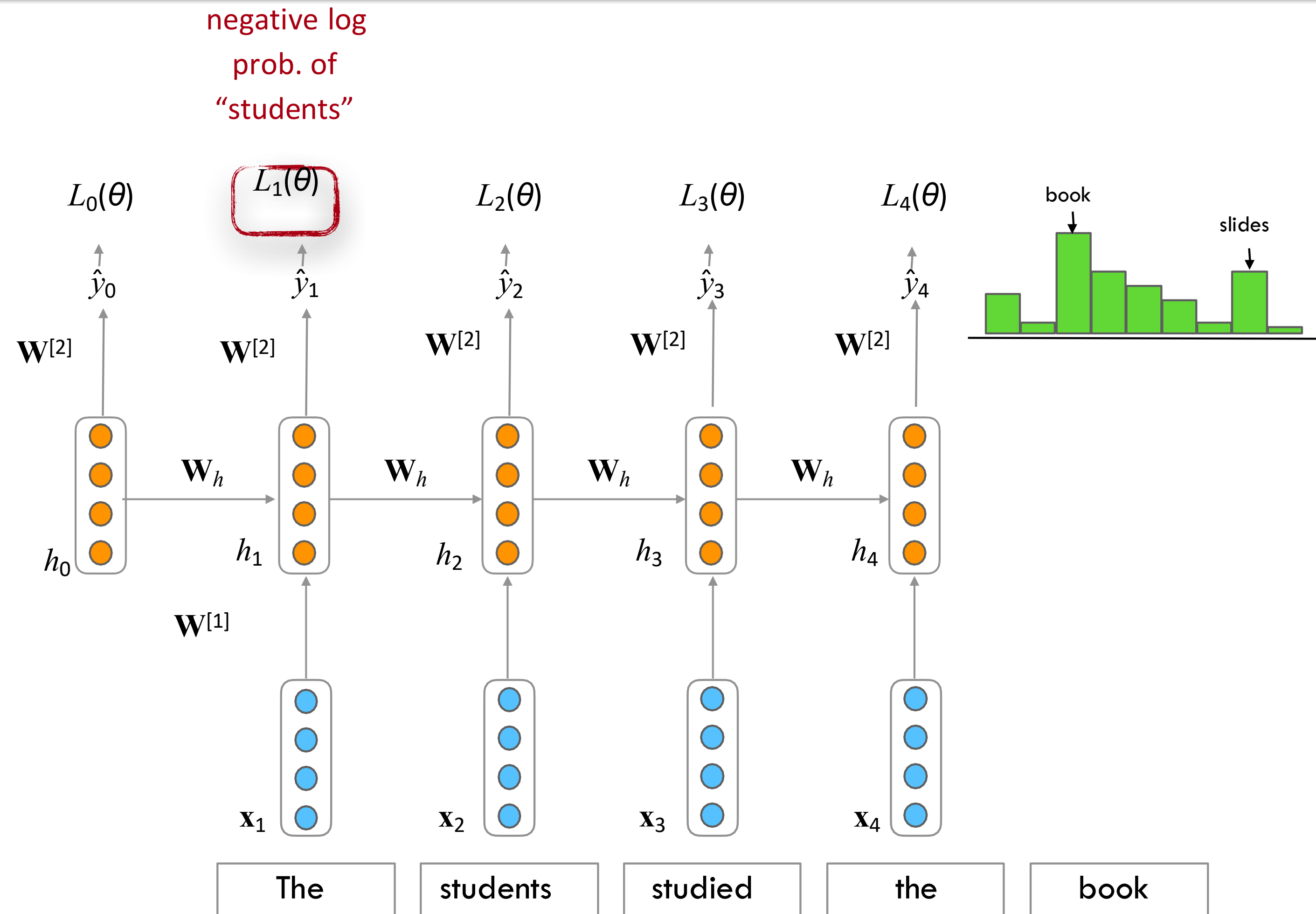
$$L(\theta) = \frac{1}{T} \sum_{t=1}^T L_{CE}(\hat{y}_t, y_t)$$

negative log
prob. of "The"

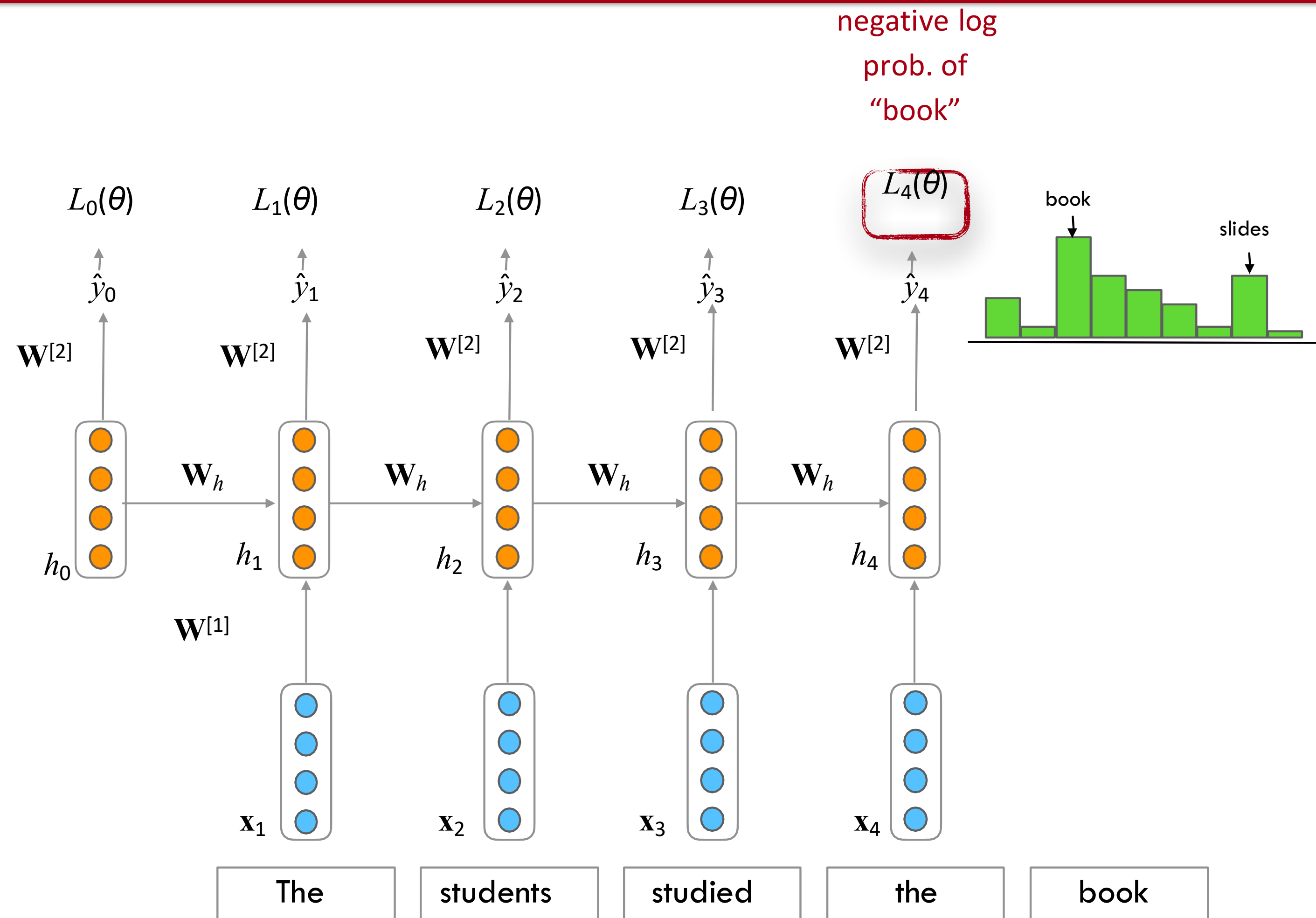
Loss



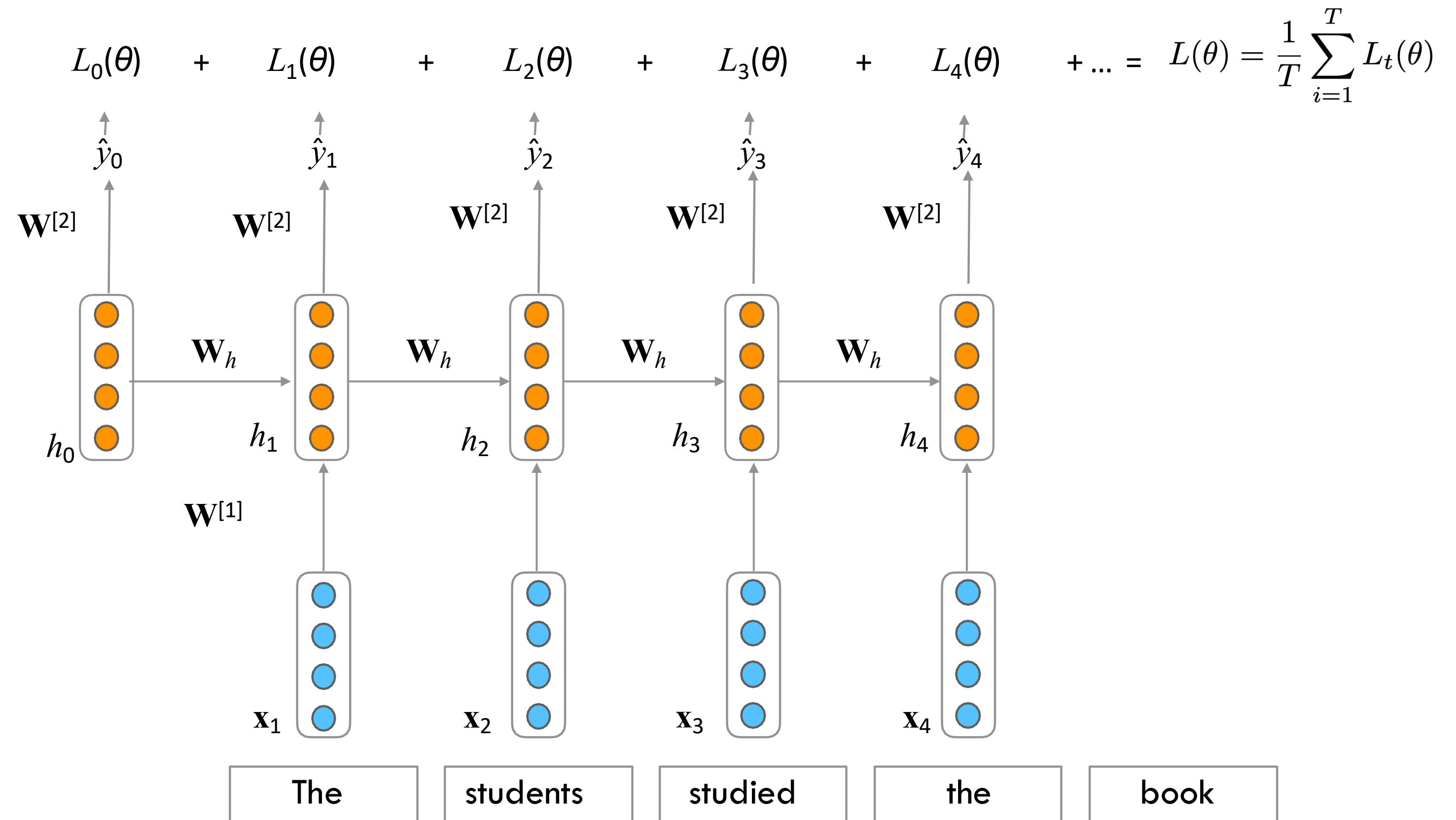
Loss



Loss



Loss



RNNs vs. Other LMs

Table 2. *Comparison of different neural network architectures on Penn Corpus (1M words) and Switchboard (4M words).*

	Penn Corpus		Switchboard	
Model	NN	NN+KN	NN	NN+KN
KN5 (baseline)	-	141	-	92.9
feedforward NN	141	118	85.1	77.5
RNN trained by BP	137	113	81.3	75.4
RNN trained by BPTT	123	106	77.5	72.5

Practical Issues with training RNNs

- Computing loss and gradients across entire corpus is too expensive!
- Recall: mini-batch Stochastic Gradient Descent allows us to compute loss and gradients for small chunk of data, and update.
- Solution: consider chunks of text.
 - In practice, consider $x_1, x_2, \dots, x_T$ for some T as a “sentence” or “single data instance”

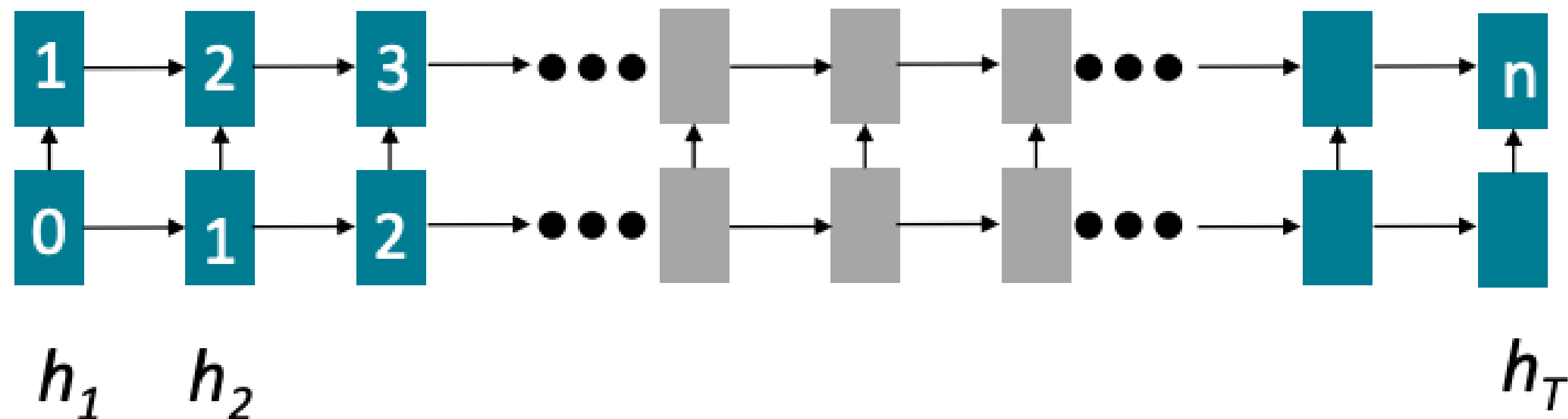
$$L(\theta) = \frac{1}{T} \sum_{t=1}^T L_{CE}(\hat{y}_t, y_t)$$

- Compute loss for a sentence (actually usually a batch of sentences), compute gradients and update weights. Repeat.

Training RNNs is hard: Parallelizability

- Forward and backward passes have $O(\text{sequence length})$ unparallelizable operations!
- GPUs can perform a bunch of independent computations at once!
- But future RNN hidden states can't be computed in full before past RNN hidden states have been computed

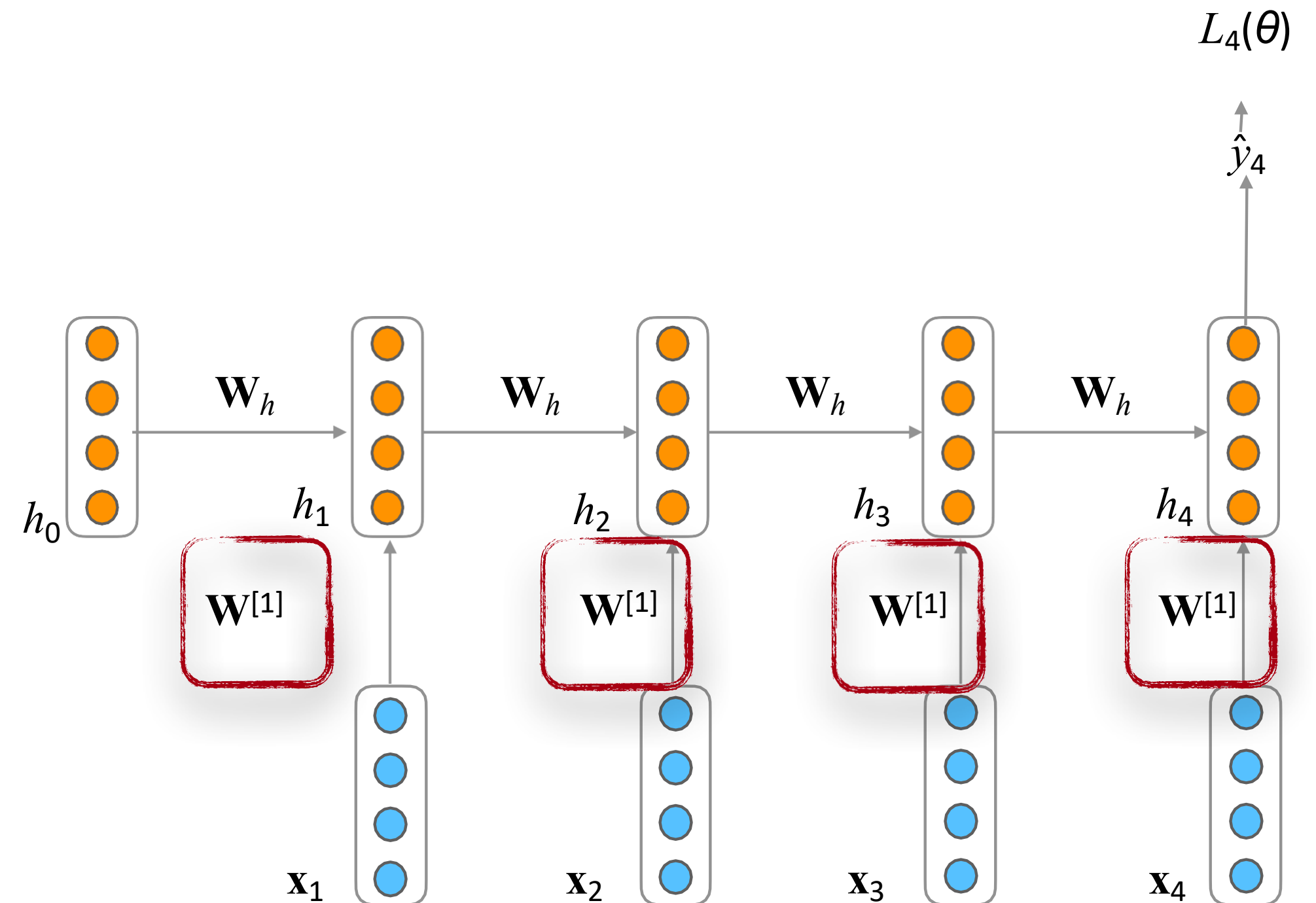
Inhibits training on very large datasets!



Numbers indicate min # of steps before a state can be computed

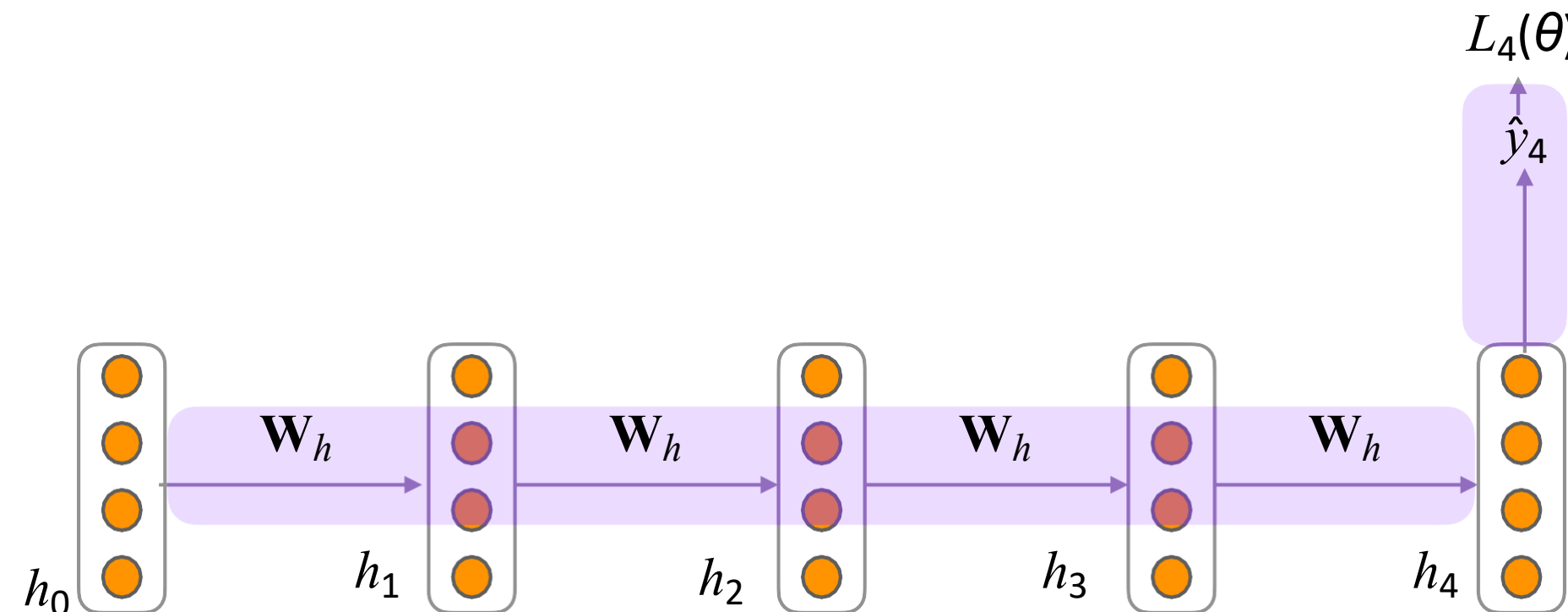
Training RNNs is hard: Gradients

- Multiply the same matrix at each time step during forward propagation
- Ideally inputs from many time steps ago can modify output
- This leads to something called the **vanishing gradient problem**



The Vanishing Gradient Problem and LSTMs

The Vanishing Gradient Problem: Intuition



When these gradients are small, the gradient signal gets smaller and smaller as it backpropagates further...

$$\begin{aligned}
 \frac{\partial L_4}{\partial h_0} &= \frac{\partial h_1}{\partial h_0} \times \frac{\partial L_4}{\partial h_1} \\
 &= \frac{\partial h_1}{\partial h_0} \times \frac{\partial h_2}{\partial h_1} \times \frac{\partial L_4}{\partial h_2} \\
 &= \frac{\partial h_1}{\partial h_0} \times \frac{\partial h_2}{\partial h_1} \times \frac{\partial h_3}{\partial h_2} \times \frac{\partial L_4}{\partial h_3} \\
 &= \frac{\partial h_1}{\partial h_0} \times \frac{\partial h_2}{\partial h_1} \times \frac{\partial h_3}{\partial h_2} \times \frac{\partial h_4}{\partial h_3} \times \frac{\partial L_4}{\partial h_4}
 \end{aligned}$$

Gradient signal from far away is lost because it's much smaller than gradient signal from close-by

The Vanishing Gradient Problem: Effects

- In practice, no long-term / long-range effects, contrary to the RNN promise
- Example language modeling task
 - To learn from this training example, the RNN-LM needs to model the dependency between “tickets” on the 7th step and the target word “tickets” at the end
- But if the gradient is small, the model can’t learn this dependency
 - So, the model is unable to predict using similar long- distance dependencies at test time
- In practice a simple RNN will only condition ~ 7 tokens back [vague rule-of-thumb]

When she tried to print her tickets, she found that the printer was out of toner. She went to the stationery store to buy more toner. It was very overpriced. After installing the toner into the printer, she finally printed her

The Vanishing Gradient Problem: Fixes

- The main problem is that it is too difficult for the RNN to learn to preserve information over many time steps
- In a vanilla RNN, the hidden state is constantly being rewritten

$$\mathbf{h}_t = \text{ReLU}(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}^{[1]} \mathbf{x}_t)$$

New design: equip an RNN with separate memory which is added to

Long Short-Term Memory RNNs (LSTMs)

- At time step t , introduces a new cell state $\mathbf{c}_t \in \mathbb{R}^d$
 - In addition to a hidden state $\mathbf{h}_t \in \mathbb{R}^d$
 - The cell stores long-term information (memory)
 - The LSTM can read, erase, and write information from the cell!
 - The cell becomes conceptually rather like RAM in a computer
- The selection of which information is erased/written/read is controlled by three corresponding gates:
 - Input gate $\mathbf{i}_t \in \mathbb{R}^d$, Output gate $\mathbf{o}_t \in \mathbb{R}^d$ and Forget gate $\mathbf{f}_t \in \mathbb{R}^d$
 - Each element of the gates can be open (1), closed (0), or somewhere in between
 - The gates are dynamic: their value is computed based on the current context

LSTMs

Given a sequence of inputs x_t , we will compute a sequence of hidden states h_t and cell states c_t

At timestep t :

Forget gate: controls what is kept vs forgotten, from previous cell state

Input gate: controls what parts of the new cell content are written to cell

Output gate: controls what parts of cell are output to hidden state

New cell content: this is the new content to be written to the cell

Cell state: erase (“forget”) some content from last cell state, and write (“input”) some new cell content

Hidden state: read (“output”) some content from the cell

Sigmoid function: all gate values are between 0 and 1

$$f^{(t)} = \sigma \left(W_f h^{(t-1)} + U_f x^{(t)} + b_f \right)$$

$$i^{(t)} = \sigma \left(W_i h^{(t-1)} + U_i x^{(t)} + b_i \right)$$

$$o^{(t)} = \sigma \left(W_o h^{(t-1)} + U_o x^{(t)} + b_o \right)$$

$$\tilde{c}^{(t)} = \tanh \left(W_c h^{(t-1)} + U_c x^{(t)} + b_c \right)$$

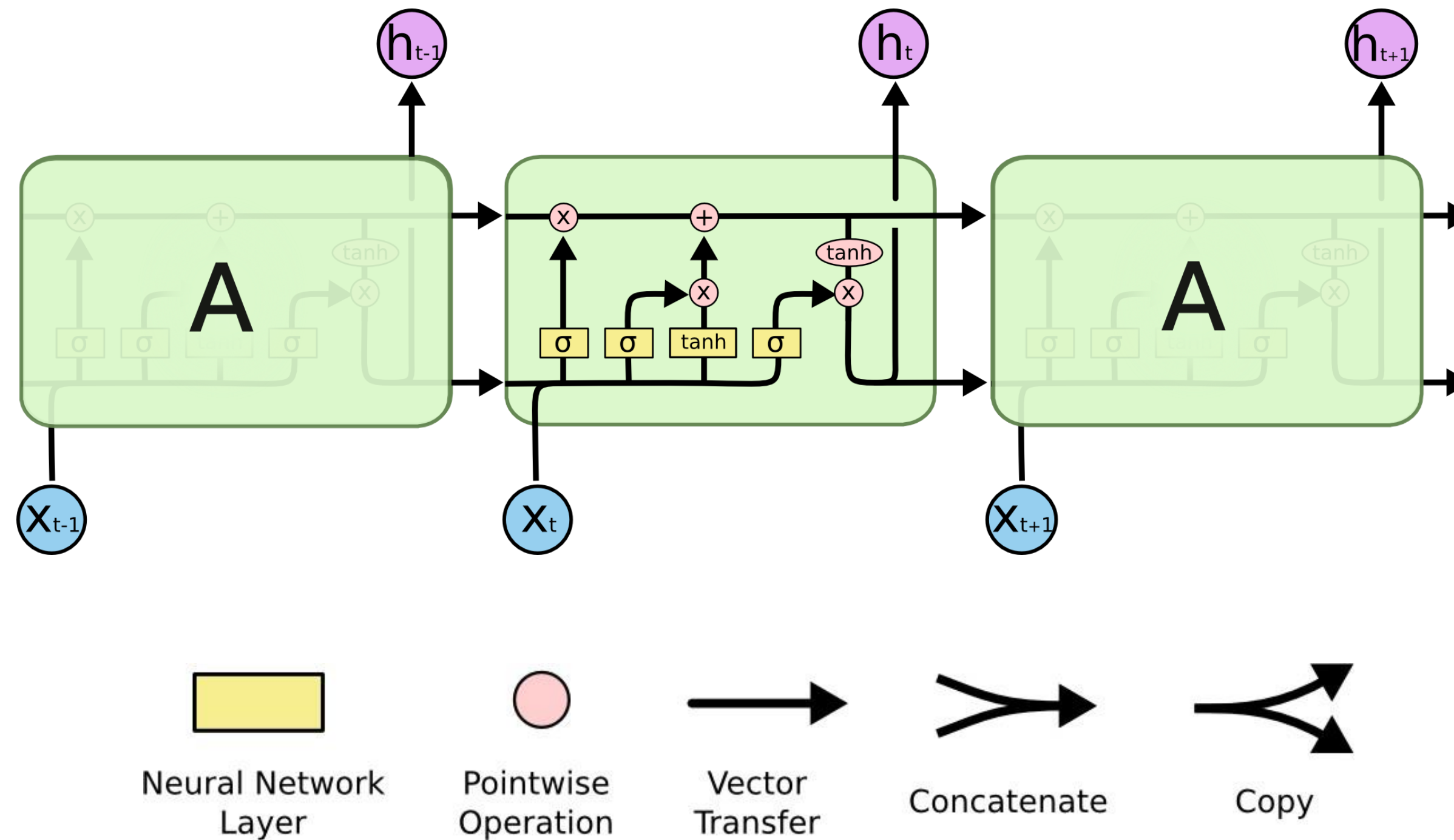
$$c^{(t)} = f^{(t)} \odot c^{(t-1)} + i^{(t)} \odot \tilde{c}^{(t)}$$

$$h^{(t)} = o^{(t)} \odot \tanh c^{(t)}$$

All these are vectors of same length n

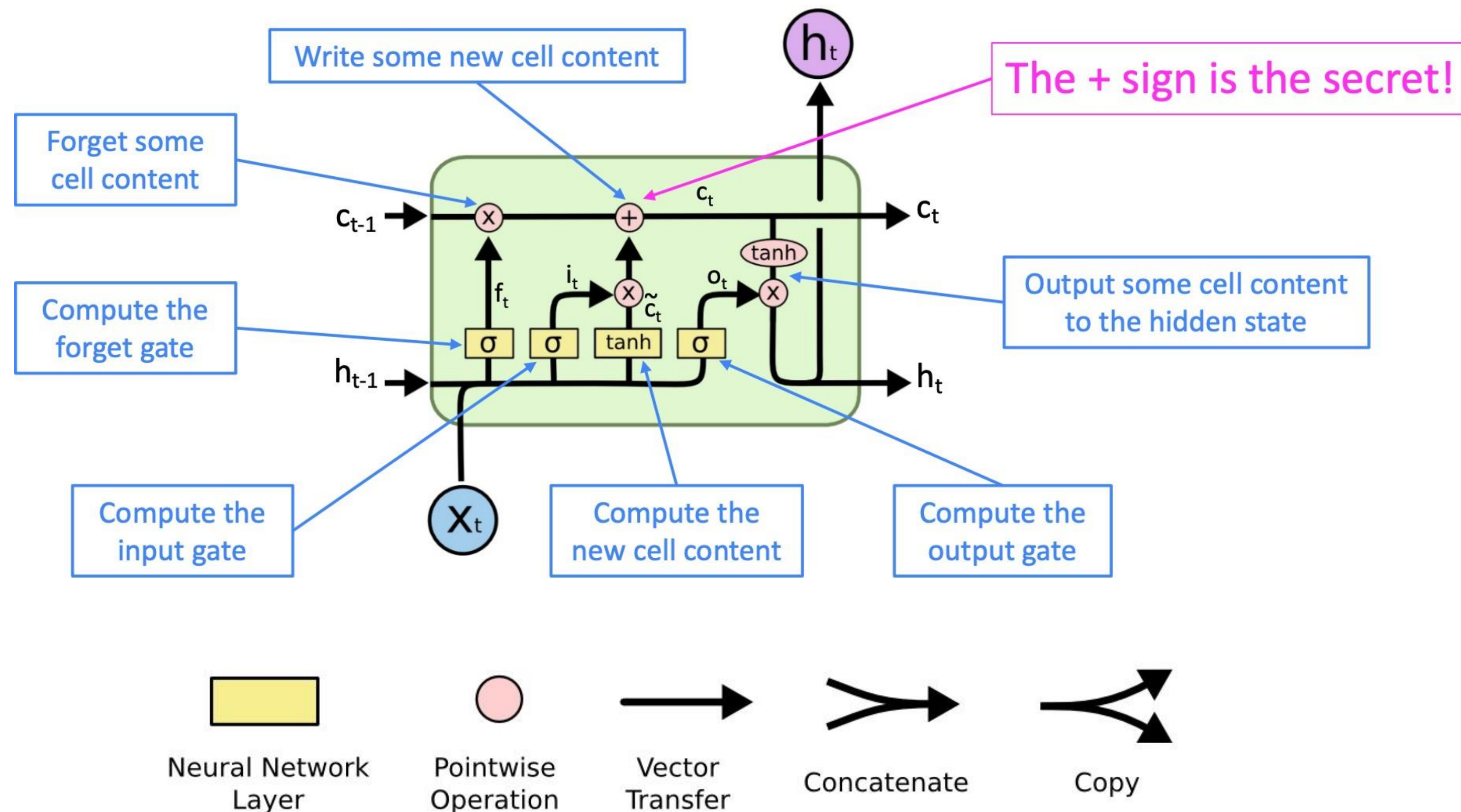
Gates are applied using element-wise (or Hadamard) product: $\odot$

LSTMs: A Visual Representation



Source: <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

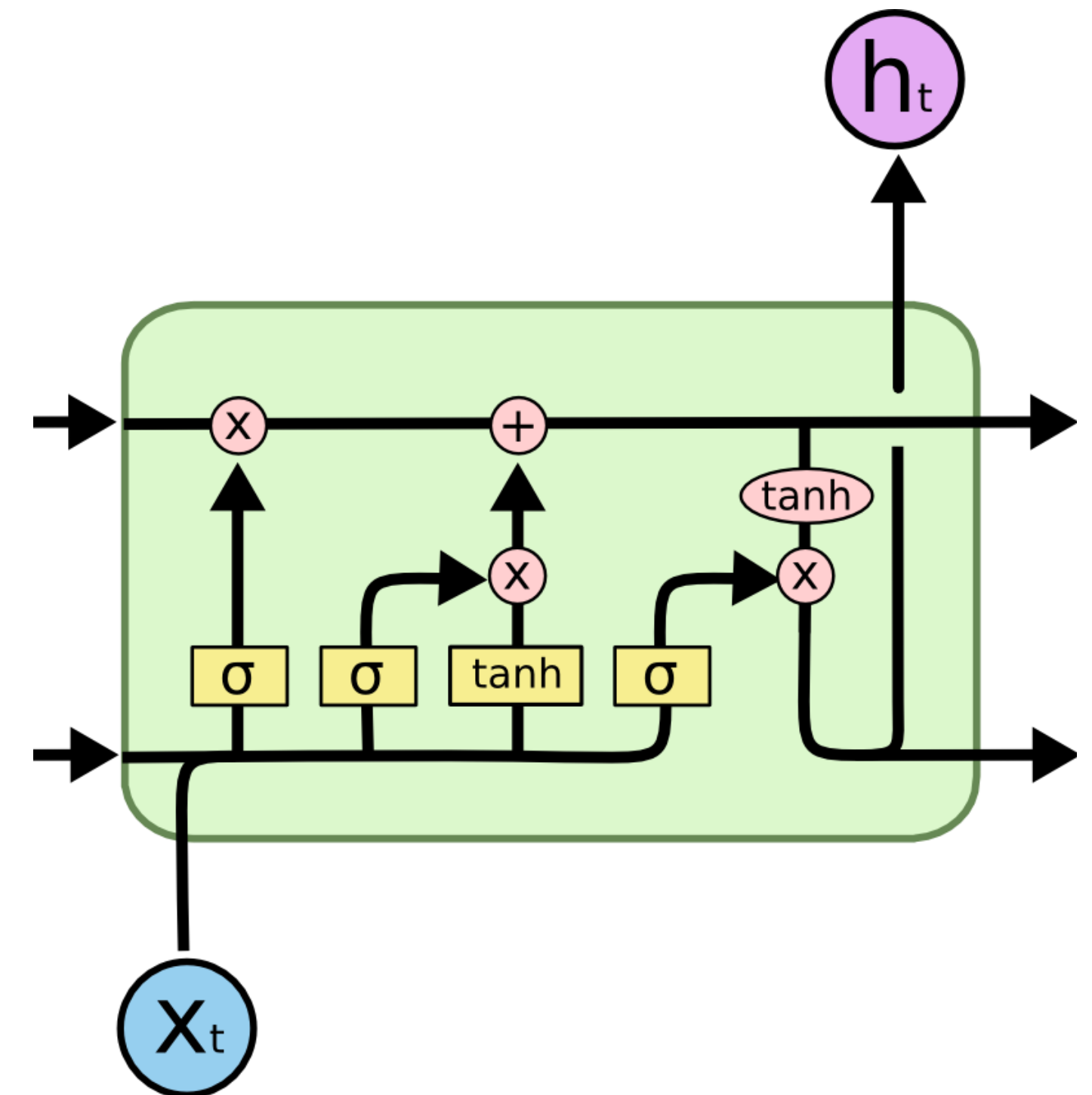
LSTMs: A Visual Representation



Source: <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

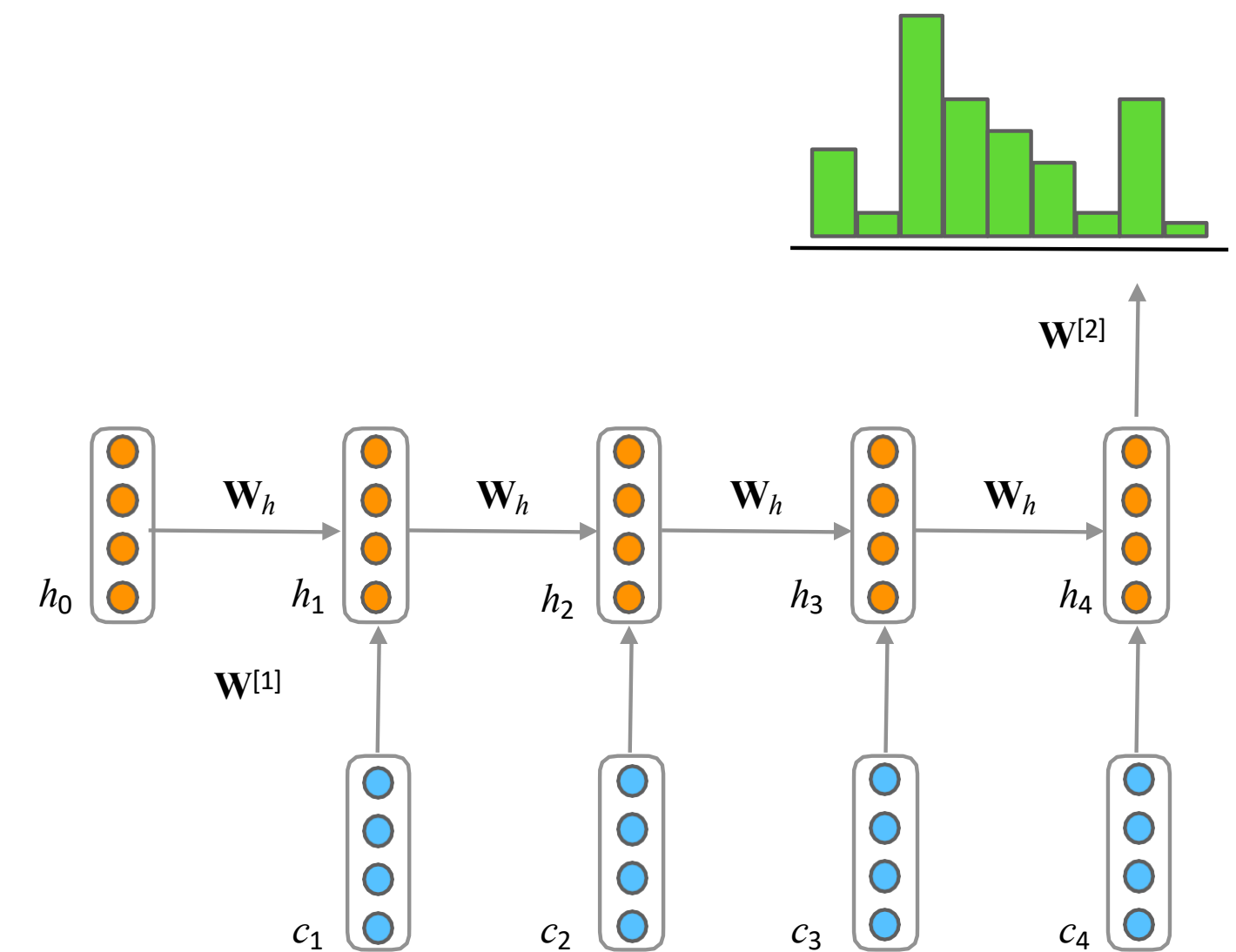
LSTMs: Summary

- The LSTM architecture makes it much easier for an RNN to preserve information over many timesteps
 - e.g., if the forget gate is set to 1 for a cell dimension and the input gate set to 0, then the information of that cell is preserved indefinitely
- In 2013–2015, LSTMs started achieving state-of-the-art results
 - Successful tasks include handwriting recognition, speech recognition, machine translation, parsing, and image captioning, as well as language models
 - LSTMs became the dominant approach for most NLP tasks



Summarizing RNNs

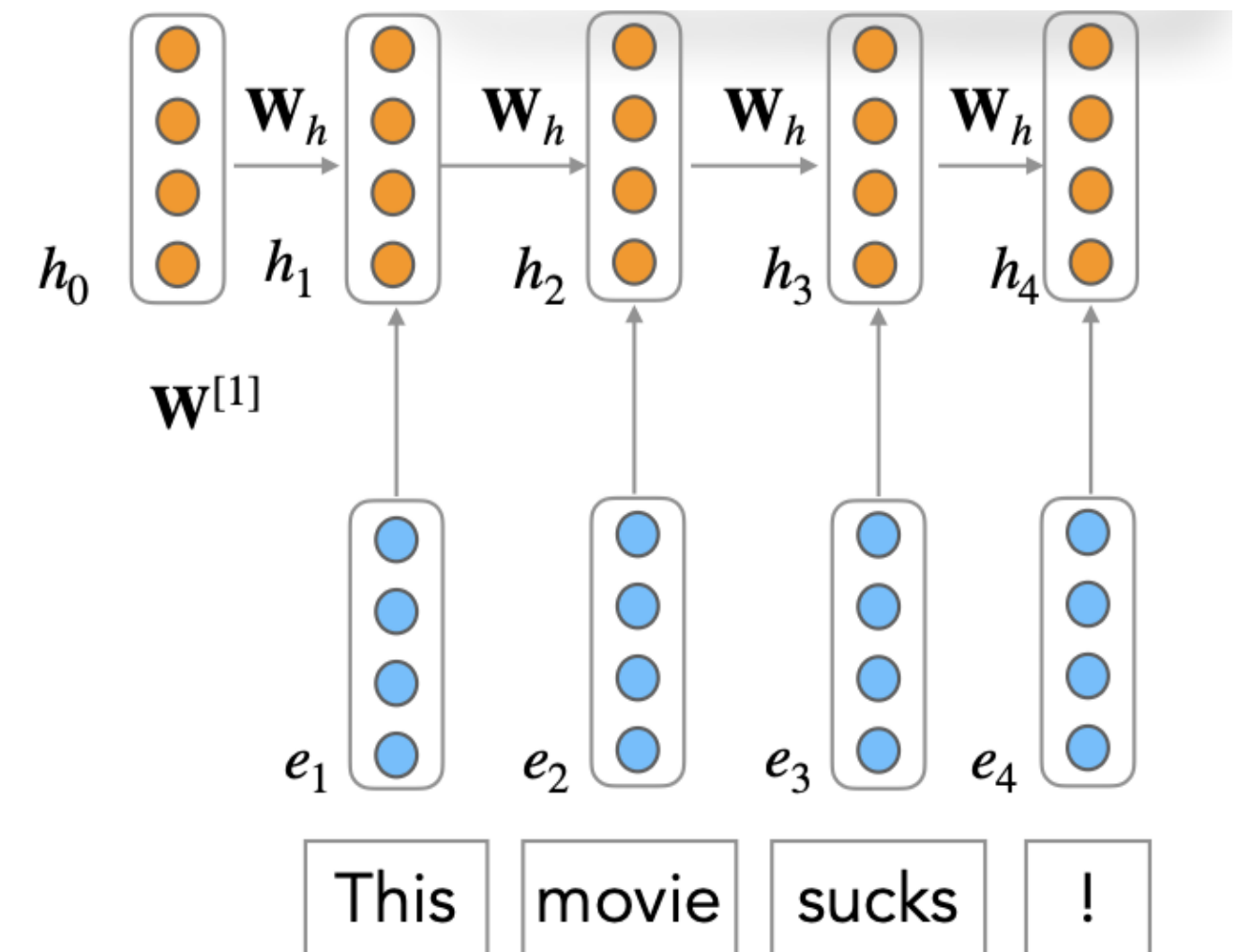
- Recurrent Neural Networks processes sequences one element at a time
- RNNs do not have (In principle)
 - the limited context problem of n-gram models
 - the fixed context limitation of feedforward LMs
 - since the hidden state can in principle represent information about all of the preceding words all the way back to the beginning of the sequence
- But training RNNs is hard (In practice)
 - Vanishing gradient problem
 - LSTMs address it by incorporating a memory



Applications of RNNs

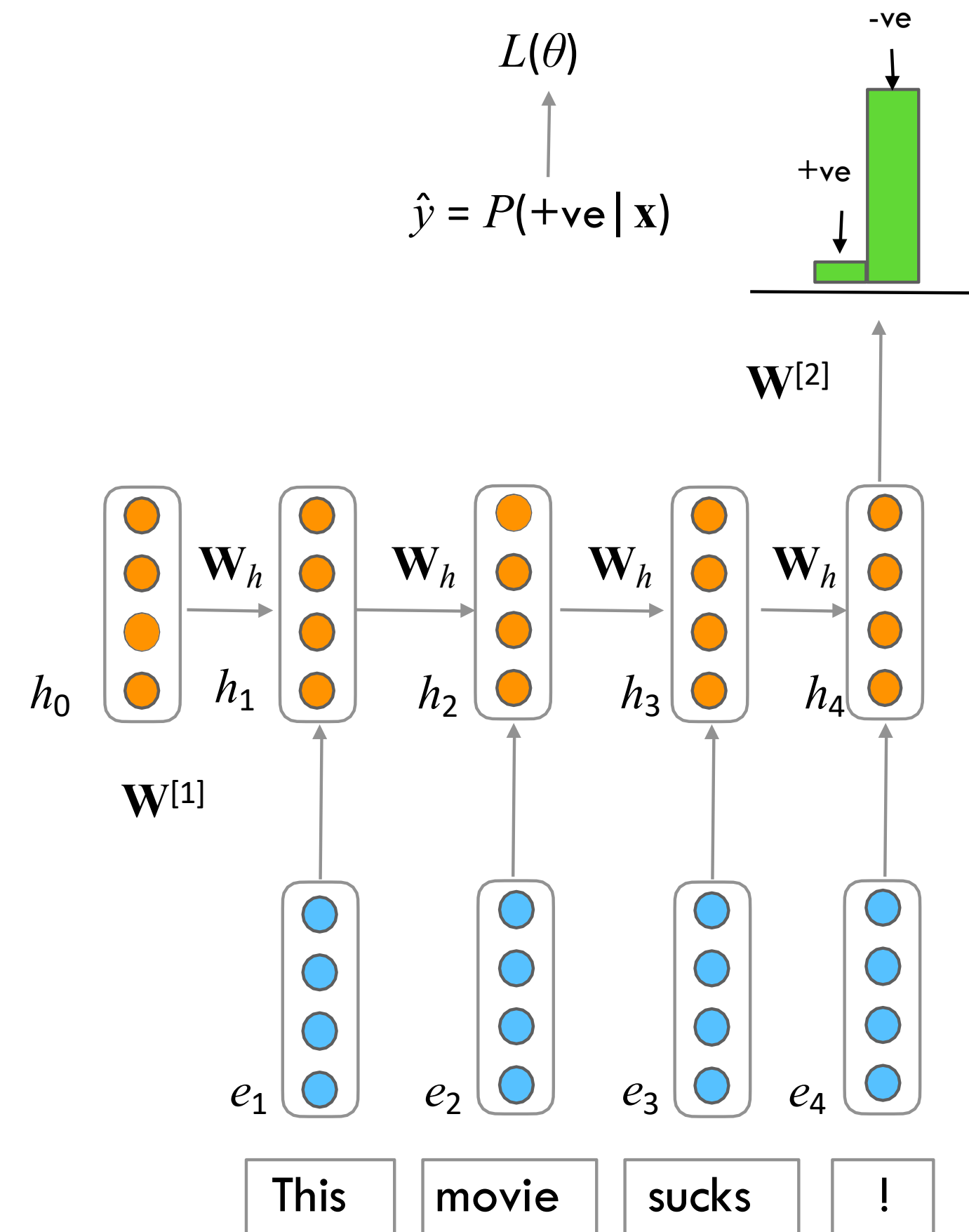
RNNs for Sequence Classification

- $\mathbf{x}$ = Entire sequence / document of length n
- $\mathbf{y}$ = (Multivariate) labels



Training RNNs for Sequence Classification

- Don't need intermediate outputs for the words in the sequence preceding the last element
- Loss function used to train the weights in the network is based entirely on the final text classification task
 - Cross-entropy loss
- Backprop: error signal from the classification is backpropagated all the way through the weights in the feedforward classifier through, to its input, and then through to the three sets of weights in the RNN



Generation with RNNLMs

Remember sampling from n-gram LMs?

- Similar to sampling from n-gram LMs
- First randomly sample a word to begin a sequence based on its suitability as the start of a sequence
- Then continue to sample words conditioned on our previous choices until
 - we reach a pre-determined length,
 - or an end of sequence token is generated

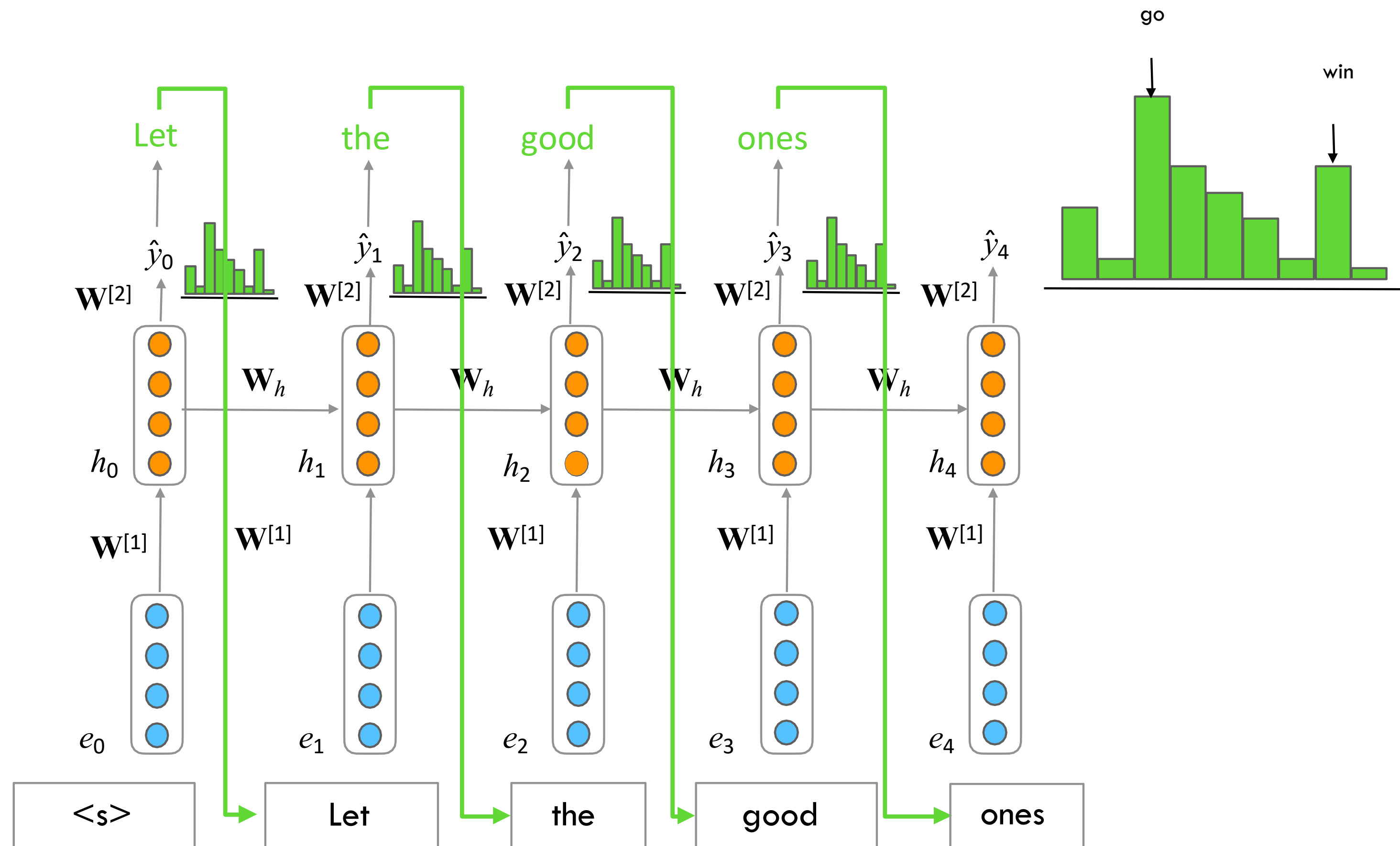
1. Choose a random bigram ($\langle s \rangle$, w_0) according to its probability
2. Now choose a random bigram (w_t , w_{t+1}) according to its probability...and so on until we choose $\langle /s \rangle$

```
<s> I
    I want
      want to
        to eat
          eat Chinese
            Chinese food
              food </s>

I want to eat Chinese food
```

$\arg \max \hat{y}_i$

$$\hat{y}_4 = P(x_5 | \text{Let the good ones})$$



Generation with RNNLMs

- Sample a word in the output from the softmax distribution that results from using the beginning of sentence marker, $\langle s \rangle$, as the first input.
- Use the word embedding for that first word as the input to the network at the next time step, and then sample the next word in the same fashion.
- Continue generating until the end of sentence marker, $\langle /s \rangle$, is sampled or a fixed length limit is reached.

Repeated sampling of the next word conditioned on previous choices

Autoregressive Generation